Training-based versus training-free differential privacy for data synthesis

Mehak Kapur
mekapur@ucsd.edu

Hana Tjendrawasi
htjendrawasi@ucsd.edu

Jason Tran
jat037@ucsd.edu

Phuc Tran
pct001@ucsd.edu

Yu-Xiang Wang
yuxiangw@ucsd.edu

Abstract

Differentially private synthetic data generation promises to resolve the tension between data utility and individual privacy, enabling the release of datasets that preserve the statistical properties analysts need while bounding what any adversary can learn about a single record. Two paradigms have emerged to fulfill this promise: (1) training-based methods, which inject calibrated noise during model optimization and couple privacy to the learning process itself, and (2) training-free methods, which leverage foundation models through black-box API access and achieve privacy through selection mechanisms that never touch model parameters.

Both paradigms have demonstrated success on image and text benchmarks, yet their behavior on realistic, multi-table relational data remains largely unexplored. We investigate both approaches on Intel's Driver and Client Applications (DCA) telemetry corpus, evaluating against a benchmark of 21 analytical SQL queries representative of production business intelligence workloads.

Code: https://github.com/mekapur/DSC180B-Q2

1 Introduction . . . 3
1.1 Motivation . . . 3
1.2 Prior work . . . 3
2 Data and problem statement . . . 5
2.1 Intel DCA telemetry data . . . 5
2.2 Analytical query workload . . . 7
2.3 Query inventory . . . 7
2.4 Formal benchmark definition . . . 9
2.5 Research questions . . . 10
3 Methods . . . 12
3.1 Training-based synthesis, DP-SGD and VAE . . . 12
3.2 Training-free synthesis, Private Evolution . . . 15
3.3 Per-table baselines . . . 18
3.4 Privacy budget summary . . . 20
4 Results . . . 21
4.1 Evaluation metrics . . . 21
4.2 DP-SGD and VAE results . . . 22
4.3 Per-table DP-SGD results . . . 24
4.4 MST results . . . 25
4.5 Private Evolution results . . . 26
4.6 Cross-method comparison . . . 28
4.7 Additive experiments . . . 33
5 Discussion . . . 34
5.1 Sensitivity to evaluation thresholds . . . 36
6 Conclusion . . . 37
References . . . 38
A Project proposal . . . 39
A.1 Abstract . . . 39
A.2 Motivation . . . 39
A.3 Data and problem statement . . . 41
B Contributions . . . 46

1 Introduction

1.1 Motivation

Organizations routinely collect detailed telemetry from their products to drive business decisions. Engineers use it to diagnose failures, product teams analyze usage trends, and analysts extract insights that shape product roadmaps. The same granularity that makes telemetry valuable also makes it sensitive. Browsing patterns, work schedules, and device fingerprints can re-identify specific individuals even after conventional anonymization.

Privacy regulations such as GDPR and CCPA, along with institutional policies, increasingly restrict how such data can be stored, shared, and analyzed. The resulting tension between data utility and privacy protection motivates the development of *synthetic data generation*, producing artificial datasets that preserve the statistical properties necessary for analysis while providing formal guarantees that no individual's information can be recovered.

Two paradigms have emerged for generating synthetic data with rigorous privacy guarantees. *Training-based methods* optimize generative models under constraints that bound individual influence, adding calibrated noise during the training process. *Training-free methods* leverage pre-trained foundation models through black-box API access, achieving privacy through carefully designed selection mechanisms rather than private optimization. Both approaches have demonstrated success in isolation on image and text benchmarks, yet no comprehensive comparison exists under controlled experimental conditions with realistic analytical workloads on multi-table relational data.

This project provides that comparison. We implement both paradigms on Intel's Driver and Client Applications (DCA) telemetry corpus and evaluate them against a benchmark of 21 SQL queries representative of production analytics, measuring which approach better preserves query fidelity under equivalent privacy budgets.

1.2 Prior work

Differential privacy, introduced by Dwork and Roth (2014), provides the foundational framework for our privacy guarantees. A randomized mechanism $\mathcal{M}$ is (ε, δ)-differentially private if for all neighboring databases D, D' differing by one record and all measurable output sets S:

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\varepsilon} \Pr[\mathcal{M}(D') \in S] + \delta. \tag{1}$$

The parameter ε controls the privacy-utility tradeoff. Smaller values confer stronger privacy at the cost of noisier outputs. The slack δ permits rare violations and must remain negligible relative to the database size. The work established key mechanisms (Laplace, Gaussian, exponential) for releasing numeric queries, along with composition theorems demonstrating that privacy loss accumulates across multiple analyses.

Abadi et al. (2016) extended differential privacy to deep learning through DP-SGD. Standard gradient descent leaks information through the unbounded influence any single train-

ing example can exert on model parameters. DP-SGD bounds this influence by clipping each per-sample gradient to a fixed ℓ_2 norm and injecting calibrated Gaussian noise to mask individual contributions. The accompanying *moments accountant* yields substantially tighter privacy bounds than naive composition, enabling practical deep learning under modest privacy budgets.

Ghalebikesabi et al. (2023) demonstrated that fine-tuning diffusion models with DP-SGD generates synthetic images of reasonable quality, though their approach requires substantial privacy budgets ($\varepsilon \approx 32$ for CIFAR-10). This motivates the search for methods that achieve comparable fidelity at lower ε.

Lin et al. (2025) introduced Private Evolution (PE), a fundamentally different paradigm that avoids model training entirely. PE operates through black-box API access to pre-trained foundation models, iteratively evolving synthetic samples by computing differentially private nearest-neighbor histograms. Each private data point votes for its nearest synthetic candidate. Gaussian noise is added to the vote histogram, and candidates are resampled according to the noisy distribution. PE achieved FID ≤ 7.9 at $\varepsilon = 0.67$ on CIFAR-10, a substantial improvement over DP-Diffusion in both privacy cost and output quality. The privacy analysis is straightforward. Each iteration releases one Gaussian mechanism, and T iterations compose to a single mechanism with noise multiplier $\sigma/\sqrt{T}$.

Xie et al. (2024) extended PE to text through Aug-PE, introducing fill-in-the-blanks variation, adaptive text lengths, and rank-based selection. Aug-PE with GPT-3.5 outperformed DP-finetuning baselines at the same privacy budget while running 12–66× faster.

Swanberg et al. (2025) adapted PE for tabular data using a workload-aware distance function that measures proximity in terms of query-relevant predicates rather than raw feature similarity. Their central finding is negative. API access to large language models does not yet improve differentially private tabular synthesis beyond established marginal-based baselines such as MST and JAM. This result informs our expectations for applying PE to the DCA telemetry corpus.

2 Data and problem statement

2.1 Intel DCA telemetry data

Our investigation uses the Intel Driver and Client Applications (DCA) telemetry dataset, a large-scale collection of system-level signals from Windows client machines. The full corpus comprises approximately 115 tables in the `university_prod` schema (9.1 TB total), organized around a globally unique identifier (`guid`) assigned to each client system. Raw event tables record per-interval measurements (hourly or daily) across power, thermal, network, application, and browsing domains. Individual tables range from tens of millions to billions of rows.

The 24 benchmark queries do not reference the raw event tables directly. They reference a `reporting` schema of pre-aggregated views that Intel analysts built on top of the raw data. These reporting tables aggregate per-event measurements to per-`guid` summaries (or per-`guid`-per-day, depending on the table). Since the reporting views are not distributed with the raw data, we reconstruct them from the raw tables using DuckDB aggregation scripts that follow Intel's documented ETL logic.

We work with a sample of the full corpus. Each raw table in the repository is split into multiple partition files of varying size, and we download a single partition per table (the first available file, typically between 1 and 5 GiB). Several tables are supplemented or replaced entirely by pre-aggregated files from a December 2024 update that Intel added to the repository. The `system_sysinfo_unique_normalized` anchor table is downloaded in full (1,000,000 `guids`). All event tables are filtered to `guids` present in this anchor, ensuring a coherent sample. Total data on disk is approximately 20.7 GiB.

Table 1 lists the 19 reporting tables we construct, organized by category. The `guid` coverage column reports how many of the 1,000,000 anchor `guids` have at least one nonzero entry in each table. Coverage varies by two orders of magnitude. The web browsing pivot table covers 512,077 `guids` (51.2%), while the PSYS RAP power metric covers only 816 (0.08%). This heterogeneous coverage is a defining characteristic of the dataset and, as we show in Section 4, a primary source of difficulty for synthesis.

Table 1: The 19 reporting tables constructed from raw DCA telemetry data.

Reporting table	Category	Raw source	Rows	Guids
`system_sysinfo_unique_normalized`	Metadata	Direct copy	1,000,000	1,000,000
`system_cpu_metadata`	Metadata	Dec. 2024 update	1,000,000	1,000,000
`system_os_codename_history`	Metadata	Dec. 2024 update	639,223	—
`system_hw_pkg_power`	Power/thermal	`hw_metric_stats`	318,791	∼800
`system_psys_rap_watts`	Power/thermal	`hw_metric_stats`	4,846	816

Continued on next page

Reporting table	Category	Raw source	Rows	Guids
system_pkg_C0	Power/thermal	hw_metric_stats	945,500	8,943
system_pkg_avg_freq_mhz	Power/thermal	hw_metric_stats	12,844	613
system_pkg_temp_centigrade	Power/thermal	hw_metric_stats	13,091	622
system_batt_dc_events	Battery	Dec. 2024 update	∼49,000	—
system_on_off_suspend_time_day	Battery	Dec. 2024 update	1,582,017	—
system_frgnd_apps_types	Application	Dec. 2024 update	56,755,998	55,830
system_userwait	Application	userwait_v2	175,223,880	38,142
system_web_cat_pivot_duration	Browsing	web_cat_pivot	512,077	512,077
system_web_cat_usage	Browsing	web_cat_usage_v2	21,354,922	64,276
system_network_consumption	Network	os_network_consumption_v2	121,843,286	37,224
system_memory_utilization	Memory	os_memsam_avail_percent	21,688,089	69,552
system_display_devices	Display	Dec. 2024 update	220,997,262	209,239
system_mods_top_blocker_hist	Sleep study	Dec. 2024 update	92,460,980	—
system_mods_power_consumption	Sleep study	Dec. 2024 update (stub)	10,000	1

The anchor table (system_sysinfo_unique_normalized) provides static client attributes such as chassis type, country, OEM, RAM capacity, CPU family and generation, processor number, operating system, and a derived persona classification. Every benchmark query that segments by demographic or geographic attributes joins against this table.

The five power and thermal tables are all derived from a single raw source (hw_metric_stats) by filtering on the name column (e.g., HW::PACKAGE:IA_POWER:WATTS: for package power, HW::PACKAGE:C0_RESIDENCY:PERCENT: for C0 residency). Each provides per-guid weighted averages and sample counts. Coverage is sparse. The C0 metric has 8,943 guids, while PSYS RAP, frequency, and temperature each cover fewer than 1,000.

The application tables contain per-process and per-application breakdowns. system_userwait records wait events (duration > 1 second) with the offending process name, wait type, and AC/DC power state. system_frgnd_apps_types records foreground application usage by executable name, application type, and daily focal screen time.

The browsing tables take two forms. system_web_cat_pivot_duration is a wide table with one row per guid and 28 browsing-category columns (education, finance, gaming, mail, news, social media, etc.), each recording total duration. system_web_cat_usage provides per-browser statistics (chrome, edge, firefox) with system count, instance count, and duration.

The `system_mods_power_consumption` table is a stub. Intel's December 2024 update contains only 10,000 rows from a single `guid`. Three benchmark queries reference this table. They execute and produce rankings, but the results reflect one client's power profile rather than population-level statistics. We retain these queries for pipeline completeness but exclude them from quantitative evaluation.

2.2 Analytical query workload

The practical utility of synthetic telemetry data is determined by its ability to support real analytical workloads. We operationalize this through a benchmark suite of 21 SQL queries developed by Intel analysts, spanning five categories:

1. *Aggregate statistics with joins* (6 queries): weighted averages across multiple tables joined on `guid`, testing whether cross-table correlations survive synthesis.
2. *Ranked top-k* (7 queries): window functions producing ranked lists of applications, processes, or browsers, testing whether relative orderings are preserved.
3. *Geographic and demographic breakdowns* (4 queries): segmentation by country, processor generation, or persona, testing preservation of conditional distributions.
4. *Histograms and distributions* (2 queries): binned aggregations testing whether distributional shapes survive synthesis.
5. *Complex multi-way pivots* (2 queries): high-dimensional joint distributions across browsing categories, devices, or user segments.

2.3 Query inventory

Table 2 lists all 21 feasible queries in the benchmark. Although the original query list contains 24 queries, three additional queries are permanently infeasible because they require the `system_mods_power_consumption` table, for which no viable data source exists in the DCA corpus available to us.[1]

[1]The three infeasible queries are `ranked_process_classifications`, `top_10_processes_per_user_id_ranked_by_total_power_consumption`, and `top_20_most_power_consuming_processes_by_avg_power_consumed`.

Table 2: Complete inventory of the 21 feasible benchmark queries, grouped by query type. "Agg+Join" denotes aggregate statistics with multi-table joins. "Top-*k*" denotes ranked lists produced by window functions. "Geo/Demo" denotes geographic or demographic breakdowns. "Histogram" denotes binned distributions. "Pivot" denotes complex multi-way pivots.

Query	Codename	Type	Description
avg_platform_power_c0_freq_temp_by_chassis	ChassisAgg	Agg+Join	Average power, C0 residency, frequency, and temperature by chassis type (5-way join).
server_exploration_1	NetAsymAgg	Agg+Join	Identify client machines sending more data than they receive, joined with sysinfo for OS and chassis.
mods_blockers_by_osname_and_codename	BlockerAgg	Agg+Join	Count distribution of modern sleep study blockers by Windows OS name and codename.
top_mods_blocker_types_durations_by_osname_and_codename	BlockerDurAgg	Agg+Join	Blocker entry counts and average durations by OS name, codename, blocker type, and activity level.
display_devices_connection_type_resolution_durations_ac_dc	DisplayAgg	Agg+Join	Display connection types with resolution and average AC/DC durations.
display_devices_vendors_percentage	VendorAgg	Agg+Join	Percentage of systems by display vendor.
most_popular_browser_in_each_country_by_system_count	BrowserRank	Top-*k*	Most popular browser per country by system count.
userwait_top_10_wait_processes	WaitRank	Top-*k*	Top 10 applications by average wait time.
userwait_top_10_wait_processes_wait_type_ac_dc	WaitModeRank	Top-*k*	Top 10 wait-time applications by wait type (app start vs. in-app) and power state.
userwait_top_20_wait_processes_compare_ac_dc_unknown_durations	WaitStateRank	Top-*k*	Top 20 wait-time applications with durations pivoted by power state (AC/DC/Unknown).
top_10_applications_by_app_type_ranked_by_focal_time	FocalRank	Top-*k*	Top 10 applications per app type by average daily focal screen time.

Continued on next page

Query	Codename	Type	Description
top_10_applications_by_app_type_ranked_by_system_count	SystemRank	Top-k	Top 10 applications per app type by distinct client count.
top_10_applications_by_app_type_ranked_by_total_detections	DetectionRank	Top-k	Top 10 applications per app type by total daily detection count.
Xeon_network_consumption	XeonGeo	Geo/Demo	Network consumption summary for Xeon vs. non-Xeon systems, by OS.
pkg_power_by_country	PowerGeo	Geo/Demo	Average CPU package power by country.
battery_power_on_geographic_summary	BatteryGeo	Geo/Demo	Battery power-on count and duration by country (min. 100 clients).
battery_on_duration_cpu_family_gen	BatteryDemo	Geo/Demo	Battery duration by CPU family and generation (min. 100 clients).
ram_utilization_histogram	RamHist	Histogram	Average RAM utilization percentage by memory capacity.
popular_browsers_by_count_usage_percentage	BrowserHist	Histogram	Percentage of systems, instances, and duration by browser.
persona_web_cat_usage_analysis	PersonaPivot	Pivot	Web browsing category duration as weighted percentages, by persona.
on_off_mods_sleep_summary_by_cpu_marketcodename_gen	SleepPivot	Pivot	On/off/sleep/MODS time percentages by CPU generation and market codename.

For the sake of brevity, we will henceforth refer to each query by its codename instead of its full name.

2.4 Formal benchmark definition

Let $\mathcal{Q} = \{q_1, \ldots, q_{21}\}$ denote our SQL query benchmark. Each query q_j maps a database instance to a result set $q_j(D) \in \mathcal{R}_j$, where $\mathcal{R}_j$ may be a scalar, vector, or table. The query discrepancy for synthetic data $\tilde{D}$ is:

$$\Delta_j(D, \tilde{D}) = d_j\big(q_j(D),\ q_j(\tilde{D})\big), \tag{2}$$

where d_j is a distance metric appropriate to the result type (relative error for scalars, Spearman's ρ for rankings, total variation for histograms).

2.4.1 Normalized error metric

A natural first attempt at an aggregate score is a binary threshold: $\text{Score}_{\text{binary}}(\tilde{D}) = \frac{1}{|\mathcal{Q}|}\sum_j \mathbf{1}[\Delta_j \leq \tau_j]$, but this has two shortcomings. First, the choice of each threshold τ_j is arbitrary and must be justified. Second, binary thresholding discards information: if two methods A and B both exceed (or both fall below) τ_j, the indicator cannot distinguish them.

We instead normalize each query discrepancy by the *natural sampling variation* of the non-private data itself. We randomly split the real database D into two disjoint halves D_1, D_2 of equal size and compute the baseline discrepancy $\Delta_j(D_1, D_2) = d_j(q_j(D_1), q_j(D_2))$. The *normalized error* for query j is then:

$$\hat{\Delta}_j(\tilde{D}) = \frac{\Delta_j(D, \tilde{D})}{|\Delta_j(D_1, D_2)|}, \quad |\Delta_j(D_1, D_2)| > 0, \tag{3}$$

where the denominator $|\Delta_j(D_1, D_2)|$ is the natural sampling variation. When D_1 and D_2 produce identical query results for a particular metric column (i.e., $|\Delta_j(D_1, D_2)| = 0$), the normalized error is undefined because there is no natural variation to calibrate against. We exclude such metrics from the normalized error computation rather than using a small stabilizer η in the denominator, since an η-stabilized ratio (e.g., $0.5/10^{-8} = 5 \times 10^7$) would produce astronomically large values that dominate summary statistics without conveying meaningful information about synthesis quality. In practice, this affects 18 out of 282 metric measurements (6.4%), primarily Jaccard group-coverage scores where the two random halves produce identical group-key sets. This ratio has a natural interpretation: $\hat{\Delta}_j = 1$ means the synthetic error equals the variation expected from simply halving the real dataset; $\hat{\Delta}_j < 1$ means the synthetic data is closer to the full real data than a random half-sample would be; $\hat{\Delta}_j \gg 1$ means the synthesis error far exceeds natural variation. The aggregate benchmark score is:

$$\text{Score}(\tilde{D}) = \frac{1}{|\mathcal{Q}|}\sum_{j=1}^{|\mathcal{Q}|} \hat{\Delta}_j(\tilde{D}). \tag{4}$$

Lower scores indicate better overall fidelity. Because the normalization is continuous, this metric can distinguish methods even when both are above or below any fixed threshold.

We compute this normalized error by randomly splitting the 1,000,000 anchor `guids` into two disjoint halves D_1 and D_2 (500,000 each, seed 42), filtering all 19 reporting tables by the split, running the full benchmark on each half, and evaluating D_1 vs. D_2 with the same metric suite used for synthetic evaluation. The resulting per-metric baselines $|\Delta_j(D_1, D_2)|$ are then used to normalize each method's errors. Table 13 reports the results. We also retain binary pass rates at the thresholds RE ≤ 0.25, TV ≤ 0.15, and $\rho \geq 0.5$ for compatibility with prior analysis.

2.5 Research questions

Given the 21-query benchmark and matched privacy budgets, we investigate the following.

1. Under matched (ε, δ), which method achieves higher benchmark scores? Which query types exhibit the largest discrepancy?
2. Does error compound across multi-table joins, or does the synthesis mechanism preserve joint distributions adequately?
3. Which method better preserves minority class frequencies (prevalence $< 5\%$)?
4. Do classifiers trained on synthetic data achieve comparable accuracy to those trained on real data?
5. What are the wall-clock time and resource requirements for each method?

3 Methods

3.1 Training-based synthesis, DP-SGD and VAE

The unit of synthesis is the `guid`. Each `guid` represents one client system, and the privacy guarantee is per-`guid`, meaning neighboring databases differ by adding or removing all rows associated with a single device.

Synthesizing each reporting table independently would destroy cross-table correlations. Most benchmark queries join multiple tables on `guid`, and if synthetic tables share no relationship, joins produce either zero matches (mismatched `guids`) or random associations (correct `guids` but uncorrelated attributes). Queries measuring cross-table relationships (e.g., "average network consumption by chassis type") would return meaningless results.

We therefore construct a single wide table with one row per `guid` containing all attributes and pre-aggregated metrics from every reporting table. For each reporting table, we compute `guid`-level aggregations of the columns referenced by benchmark queries. Multi-row-per-`guid` tables (e.g., web browsing by category) are pivoted into separate columns per category. All aggregations are LEFT JOINed onto the sysinfo anchor table (1,000,000 `guids`).

The resulting wide table has 1,000,000 rows and 70 columns, comprising 9 categorical attributes and 59 numeric metrics plus `guid`. We encode categoricals via top-k binning ($k = 50$, with remaining values grouped into "Other") and one-hot encoding. Numeric columns are clipped at the 99.9th percentile, transformed via $\log(1+x)$ to reduce skew, and standardized with zero mean and unit variance. The final feature matrix has 1,000,000 × 307 entries (248 one-hot indicators plus 59 scaled numerics).

3.1.1 DP-VAE architecture

We train a differentially private variational autoencoder (DP-VAE) on the wide table. Let $x \in \mathbb{R}^{307}$ denote a `guid`-level feature vector. The encoder maps x through two hidden layers of 512 units each to produce a 64-dimensional mean μ and log-variance $\log \sigma^2$. We sample $z = \mu + \sigma \odot \epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$.

The decoder uses separate heads for categorical and numeric attributes:

- 9 categorical heads, each producing logits over the corresponding column's vocabulary, trained with cross-entropy loss.
- 1 numeric head mapping z to 59 outputs, trained with mean squared error.

The total loss is:

$$\mathcal{L} = \sum_{c=1}^{9} \mathrm{CE}(x_c, \hat{x}_c) + \mathrm{MSE}(x_{\mathrm{num}}, \hat{x}_{\mathrm{num}}) + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\big). \tag{5}$$

The model has 505,971 trainable parameters.

Algorithm 1: DP-VAE training with DP-SGD

Input : Wide table $X \in \mathbb{R}^{n \times 307}$, target $\varepsilon^\star$, δ, clip norm C, epochs E, batch size B
Output: Trained parameters θ, final ε
Wrap optimizer with Opacus: $\sigma \leftarrow$ `make_private_with_epsilon`$(\varepsilon^\star, \delta, E, B)$;
for *epoch* $= 1, \ldots, E$ **do**
  for *each batch* $\{x_i\}_{i=1}^{B} \subset X$ **do**
    Encode: $(\mu_i, \log \sigma_i^2) \leftarrow \text{Encoder}(x_i)$;
    Sample: $z_i \leftarrow \mu_i + \sigma_i \odot \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, I)$;
    Decode: $\hat{x}_i \leftarrow \text{Decoder}(z_i)$;
    Compute $\mathcal{L}_i = \text{CE}_i + \text{MSE}_i + \text{KL}_i$;
    Clip: $\bar{g}_i \leftarrow \nabla_\theta \mathcal{L}_i \cdot \min(1, C/\|\nabla_\theta \mathcal{L}_i\|_2)$;
    Noise: $\tilde{g} \leftarrow \frac{1}{B}\big(\sum_i \bar{g}_i + \mathcal{N}(0, \sigma^2 C^2 I)\big)$;
    Update: $\theta \leftarrow \theta - \eta \tilde{g}$;

$\varepsilon \leftarrow$ `PrivacyAccountant.get_epsilon`(δ);

3.1.2 DP-SGD training and privacy accounting

We use the Opacus library to apply DP-SGD. Rather than manually selecting a noise multiplier, we use Opacus's `make_private_with_epsilon` to automatically calibrate σ for a target budget of $\varepsilon = 4.0$ at $\delta = 10^{-5}$. Privacy loss is tracked via the Rényi differential privacy (RDP) accountant, which yields tighter bounds than basic composition.

Table 3 reports the training configuration. The final privacy expenditure is $\varepsilon = 3.996$.

Table 3: DP-VAE training configuration.

Hyperparameter	**Value**
Encoder architecture	$307 \to 512 \to 512 \to (64\mu, 64 \log \sigma^2)$
Decoder architecture	$64 \to 512 \to 512 \to 9$ categorical + 1 numeric head
Total parameters	505,971
Batch size	4,096
Epochs	20
Learning rate	10^{-3} (Adam)
Max gradient norm C	1.0
Noise multiplier σ	auto-calibrated by Opacus
Target ε	4.0
δ	10^{-5}
Final ε	3.996
Training time	359.7 min (CPU)

3.1.3 Training dynamics

Figure 1 plots the training loss and cumulative privacy budget (ε) over the 20 epochs of DP-SGD training. Because Opacus applies per-sample gradient clipping and Gaussian noise injection at every step, the per-epoch loss is noisier than in a standard (non-private) VAE. Despite this, the training loss decreases monotonically from 23.76 in epoch 1 to 17.31 in epoch 20 (a 27% reduction), indicating that the model learns signal through the privacy noise. The steepest improvement occurs in the first 5 epochs (23.76 → 19.70), after which the rate of decrease slows. By epoch 15 the loss curve is nearly flat (17.79 → 17.31 over the final 5 epochs), suggesting diminishing returns from additional training. The cumulative privacy spend increases roughly linearly, reaching $\varepsilon = 3.996$ at epoch 20, just under the target budget of $\varepsilon = 4.0$.

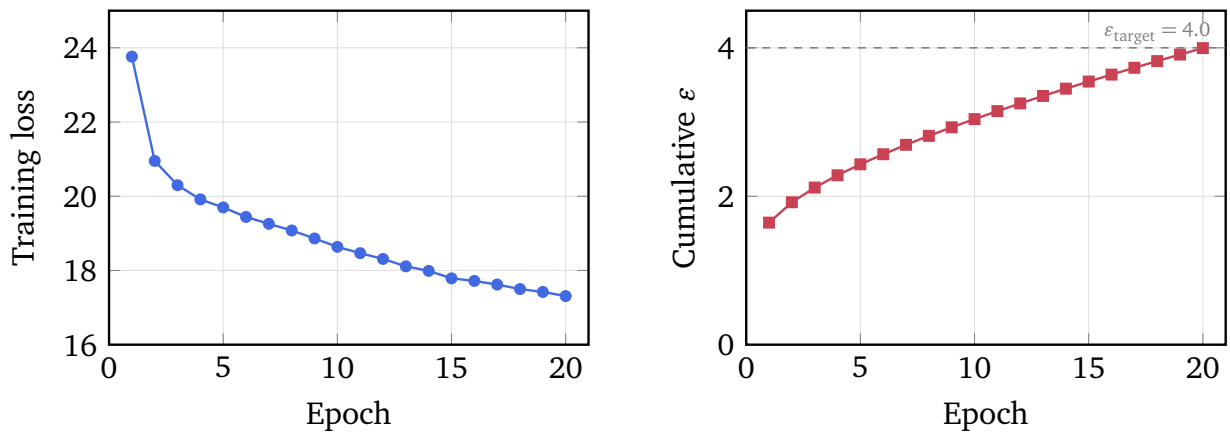

Figure 1: Wide-table DP-VAE training dynamics over 20 epochs ($\varepsilon_{\text{target}} = 4.0$, $\delta = 10^{-5}$). Left: training loss decreases monotonically from 23.76 to 17.31, with the steepest improvement in early epochs. Right: cumulative privacy budget grows approximately linearly, reaching $\varepsilon = 3.996$ at convergence. Data extracted from the saved model checkpoint.

3.1.4 Synthetic data generation and decomposition

After training, we generate 1,000,000 synthetic `guid` records:

1. Sample $z \sim \mathcal{N}(0, I_{64})$.
2. Decode to produce categorical logits and numeric outputs.
3. For each categorical column, sample from the softmax distribution over the vocabulary.
4. For each numeric column, apply the inverse transformations (de-standardize, apply $\text{expm1}(x) = e^x - 1$, clip to the observed range).

The synthetic wide table is then decomposed back into 12 reporting table schemas by selecting the relevant columns for each table and assigning synthetic `guids`. The original benchmark SQL queries execute unchanged on these synthetic reporting tables.

We evaluate 8 of the 21 feasible queries whose reporting tables are fully reconstructable

from the wide-table columns. The remaining 13 queries require per-row columns not present in the wide table (e.g., per-process breakdowns for userwait, per-application data for foreground apps, per-display data for display devices).

3.2 Training-free synthesis, Private Evolution

We implement Private Evolution adapted for tabular data, following the framework of Lin et al. (2025) with the workload-aware distance function proposed by Swanberg et al. (2025). The algorithm operates in a single iteration ($T = 1$, the optimal setting for tabular data per Swanberg et al.'s finding that subsequent iterations provide marginal gains outweighed by composition cost).

1. *Population generation.* Prompt a foundation model (OpenAI gpt-5-mini via the structured outputs API) to produce $N_{\text{synth}} \times L$ synthetic records conforming to the DCA wide-table schema (68 fields, 9 categorical as constrained strings, 59 numeric as floats). Each API call returns a batch of 5 records and uses Pydantic Structured Outputs to guarantee schema compliance.
2. *DP nearest-neighbor histogram.* Each of the $n = 1{,}000{,}000$ real records votes for its nearest synthetic candidate under the workload-aware distance. Gaussian noise $\mathcal{N}(0, \sigma^2)$ is added to the vote histogram. The sensitivity is 1, since each `guid` contributes exactly one vote.
3. *Rank-based selection.* The top N_{synth} candidates by noisy vote count are selected as the final synthetic population.

The workload-aware distance follows Swanberg et al.'s formulation:

$$d_W(x, c) = \sum_{i \in \text{cat}} w_i \cdot \mathbf{1}[x_i \neq c_i] + \sum_{j \in \text{num}} |x_j - c_j|, \tag{6}$$

where w_i weights categorical features by query frequency (e.g., `chassistype` appears in 6 queries and receives weight 6, `countryname` in 4 queries receives weight 4) and numeric features are min-max normalized before computing L1 distance.

For $T = 1$ iteration, the privacy analysis reduces to a single Gaussian mechanism. We calibrate σ via the analytic Gaussian mechanism of Balle and Wang (2018) to achieve $\varepsilon = 4.0$ at $\delta = 10^{-5}$, yielding $\sigma \approx 1.08$. With $N_{\text{synth}} = 5{,}000$ and $L = 2$, we generate 10,000 candidate records through 2,000 API calls (batch size 5), submitted via asynchronous structured output requests at 50 concurrent calls per chunk.

Given Swanberg et al.'s negative result on tabular PE (API access did not improve over marginal-based baselines), we treat this comparison as an empirical test of whether PE's advantages in the image and text domains transfer to heterogeneous, sparse telemetry data.

3.2.1 Prompt construction

The quality of the generated candidate population depends critically on how the prompt communicates the data schema and sparsity structure to the foundation model. We con-

struct each prompt in three parts: a *system instruction*, a *schema description*, and a *generation directive*. The system instruction is fixed across all calls:

```
You generate synthetic tabular data. Produce realistic, diverse
records following the schema and sparsity rules exactly.
```

The schema description is computed once from the real data and enumerates: (i) the 9 categorical columns with their top-10 most frequent values as constrained pick-lists, (ii) the RAM field (integer GB: 4/8/16/32/64), and (iii) the 58 numeric columns organized into 8 semantic groups (`net`, `mem`, `batt`, `browser`, `webcat`, `onoff`, `disp`, `hw`), each annotated with the percentage of real systems that have nonzero values in that group. Figure 2 shows a condensed example.

PE prompt (condensed)

```
Generate exactly 5 synthetic telemetry records.
Schema:
Intel telemetry. Each record represents one client system.
Categorical (pick from list): chassistype: ["Notebook", "Desktop", "2 In 1", ...]; countryname_
normalized: ["United States", "India", "Germany", ...]; os: ["Windows 11", "Windows 10", ...];
persona: ["Office/Productivity", "Gamer", ...]; cpu_family: ["Core", "Celeron", ...]; …
ram: integer GB (4/8/16/32/64), always present.
Numeric groups (all float ⩾ 0): net (3.7% of systems): net_nrs, net_received_bytes, net_sent_bytes;
mem (6.9%): mem_nrs, mem_avg_pct_used, mem_sysinfo_ram; batt (2.0%): batt_num_power_ons, batt_
duration_mins; browser (6.4%): web_chrome_duration, web_edge_duration, …; webcat (5.9%): webcat_
education, webcat_entertainment_video_streaming, … (28 cols); onoff (3.7%): onoff_on_time, …; disp
(20.9%): disp_num_displays, …; hw (<1%): psys_rap_nrs, …
RULE: each system has data in 1–2 groups only. Set all other groups to 0. Most groups are present in
<7% of systems.
Generate diverse, realistic records. Follow the sparsity rules strictly.
```

Figure 2: Condensed PE prompt sent to gpt-5-mini. The full prompt lists all 68 fields and the top-10 values for each categorical column. Each API call requests 5 records and enforces schema compliance via a Pydantic `json_schema` structured output constraint, so the model cannot produce invalid field names, out-of-vocabulary categorical values, or malformed types.

A single generated record has the following shape (most numeric fields are zero, reflecting the sparsity rule):

```
{
"chassistype": "Notebook",  "countryname_normalized": "Germany",
"os": "Windows 11",  "persona": "Office/Productivity",
"cpu_family": "Core",  "cpuname": "12th Gen Intel Core",
"cpucode": "Alder Lake",  "processornumber": "i5-1235U",
"modelvendor_normalized": "Lenovo",
"ram": 16.0,
"net_nrs": 0.0, "net_received_bytes": 0.0, "net_sent_bytes": 0.0,
"mem_nrs": 28.0, "mem_avg_pct_used": 62.3, "mem_sysinfo_ram": 16384.0,
"batt_num_power_ons": 0.0, "batt_duration_mins": 0.0,
"web_chrome_duration": 3841.2, "web_edge_duration": 0.0, …,
"webcat_education": 142.5, "webcat_productivity_other": 891.3, …,
"onoff_on_time": 0.0, …,  "disp_num_displays": 0.0, …,
"psys_rap_nrs": 0.0, …  (all hw fields = 0)
}
```


Figure 3: Example synthetic record generated by gpt-5-mini. This record has nonzero values only in the `mem` and `browser/webcat` groups (2 of 8 groups active), consistent with the sparsity rule. All 68 fields are present and type-correct due to the structured output constraint.

The key quality concern is that the foundation model's prior over categorical values diverges from the real distribution. The model over-represents common categories such as "United States," "Notebook," and "Windows 11" relative to the DCA corpus. The DP nearest-neighbor histogram (Step 2 of the algorithm) partially corrects these biases by upweighting candidates that resemble real records, but cannot compensate when entire regions of the feature space are absent from the candidate pool.

Algorithm 2: Private Evolution for tabular data ($T = 1$)

Input : Real table $X \in \mathbb{R}^{n \times d}$, target $\varepsilon^\star$, δ, population size N_{synth}, variation factor L
Output: Synthetic dataset S of size N_{synth}
Calibrate σ via analytic Gaussian mechanism (Balle and Wang 2018) for $(\varepsilon^\star, \delta, T{=}1)$;
$S_0 \leftarrow$ `RANDOM_API`$(N_{\text{synth}} \times L)$; `// generate candidates via LLM`
for $i = 1, \ldots, n$ **do**
  $j_i^\star \leftarrow \arg\min_j d_W(x_i, S_0^{(j)})$; `// nearest neighbor under workload distance`
$h_j \leftarrow |\{i : j_i^\star = j\}|$ for each candidate j; `// vote histogram`
$\tilde{h}_j \leftarrow h_j + \mathcal{N}(0, \sigma^2)$ for each j; `// add calibrated Gaussian noise`
$S \leftarrow$ top N_{synth} candidates ranked by $\tilde{h}_j$; `// rank-based selection`
return S

3.2.2 Conditional generation with DP-noised aggregates

We extend the vanilla PE pipeline with a conditional generation variant that guides the foundation model using aggregate statistics derived from the real data under differential privacy. Rather than generating records independently from the model's prior, conditional PE provides per-stratum targets that steer generation toward the true data distribution.

The approach proceeds in four phases.

1. Compute aggregate statistics per stratum from the real data, where each stratum corresponds to a unique combination of categorical values and active numeric groups.
2. Add calibrated Gaussian noise to every aggregate query via the Gaussian mechanism, spending a privacy budget of $\varepsilon_{\text{agg}} = 3.5$. We employ zero-concentrated differential privacy (zCDP) composition (Bun et al. 2016), converting the total $(\varepsilon_{\text{agg}}, \delta)$ budget to a zCDP parameter ρ_{total} and splitting ρ equally across 7 query groups (battery, memory, on/off, browser/webcat, hardware, network, and hints). This yields per-query noise scales $\sigma_g = \sqrt{k_g/(2\rho_g)}$ ranging from 24.6 (network, $k = 40$ queries) to 68.2 (browser/webcat, $k = 308$ queries), substantially tighter than the $\sigma \approx 150$ that naive advanced composition would require across all 1,056 queries jointly.
3. Use the noisy aggregates to construct per-stratum prompts that specify target counts and averages for each numeric group. Since these prompts are deterministic functions of the DP-released aggregates, this step is free post-processing under DP.
4. Generate a 2× candidate pool across all strata and select $N_{\text{synth}} = 5{,}000$ records via the DP nearest-neighbor histogram mechanism, spending $\varepsilon_{\text{hist}} = 0.5$.

The total privacy cost is $\varepsilon_{\text{agg}} + \varepsilon_{\text{hist}} = 4.0$ by basic composition, matching the vanilla PE budget. The budget split favors the aggregates over the histogram (3.5 vs. 0.5) because conditional generation already aligns the candidate pool with the target distribution, reducing the marginal value of the histogram selection step. The resulting histogram noise scale is $\sigma_{\text{hist}} \approx 7.03$, compared to $\sigma \approx 1.08$ in vanilla PE where the histogram receives the full budget.

3.3 Per-table baselines

To isolate the sparsity artifact of the wide-table approach, we implement an independent per-table synthesis baseline. Each of the 19 reporting tables is synthesized separately at $\varepsilon = 4.0$ per table.

Two tables receive dedicated DP-VAE treatment. The `system_sysinfo_unique_normalized` table (1,000,000 rows, 9 categorical and 1 numeric column) uses a VAE with encoder $d_{\text{in}} \to 256 \to 128 \to (32\mu, 32\log\sigma^2)$, batch size 4,096, 15 epochs, Adam (lr $= 10^{-3}$), KL weight 0.1, trained with Opacus at $\varepsilon = 4.0$, $\delta = 10^{-5}$. Categoricals are top-50 binned and one-hot encoded. The `system_cpu_metadata` table (1,000,000 rows, 5 categorical and 1 numeric column) uses the same architecture.

The remaining 17 tables use DP histogram synthesis. For each table, we compute the join rate (fraction of anchor `guids` with data) and generate proportionally many synthetic rows. Continuous columns are discretized into 20 equal-width bins spanning the 1st to 99th percentile, with Laplace noise (scale $= 1/\varepsilon_c$) added to bin counts and $\varepsilon_c = \varepsilon/k$ split uniformly across k columns. Categorical columns receive analogous noisy histogram sampling.

This approach sacrifices cross-table correlations by design. Synthetic `guids` in one table bear no relationship to those in another. It does avoid the zero-inflation problem because each table-level model trains only on `guids` with observed data, so zeros do not dominate the input distribution. Under basic composition, the total privacy budget is $19 \times 4.0 = 76.0$,

which is substantially weaker than the wide-table guarantee of $\varepsilon = 4.0$. This is a known limitation of per-table synthesis. Tighter accounting may reduce the effective budget and is left for future work.

Per-table synthesis also serves as a diagnostic. If it outperforms the wide-table approach on single-table queries but fails on multi-table joins, that supports the sparsity hypothesis.

3.3.1 MST baseline

As a non-neural baseline, we implement the Maximum Spanning Tree (MST) algorithm (McKenna, Miklau and Sheldon 2021) using the SmartNoise Synth library (OpenDP Contributors 2024). MST is a marginal-based method that operates in three phases:

1. *Marginal selection.* For a table with d columns, MST considers all $\binom{d}{2}$ pairwise column marginals. It scores each pair (c_i, c_j) by mutual information estimated from the joint histogram, then selects $d-1$ pairs forming a maximum spanning tree over the column graph using the exponential mechanism (McSherry and Talwar 2007). The exponential mechanism ensures that the *selection itself* is differentially private: high-mutual-information pairs are chosen with higher probability, but no exact marginal value is revealed. The privacy cost of selection is ε_{sel}, allocated from the total per-table budget.
2. *Noisy measurement.* For each of the $d-1$ selected column pairs, MST materializes the full two-way contingency table and adds calibrated Gaussian noise (sensitivity $= 1$ per cell, since each row contributes to exactly one cell). The remaining budget $\varepsilon_{\text{meas}} = \varepsilon - \varepsilon_{\text{sel}}$ is split equally across all $d-1$ measurements. For a table with 10 columns, this means 9 noisy two-way marginals, each receiving $\varepsilon_{\text{meas}}/9$ of the measurement budget.
3. *Synthetic generation via Private-PGM.* The noisy marginals define a graphical model whose structure is the selected tree. MST uses the Private-PGM algorithm (McKenna, Sheldon and Miklau 2019) to find a distribution over records that is maximally consistent with all noisy marginals simultaneously. Private-PGM solves this as a convex optimization problem (iterative proportional fitting on the junction tree), then samples n synthetic records from the fitted distribution. Because the tree structure is sparse, the graphical model decomposes into tractable local computations even for tables with many columns.

MST requires all columns to have finite cardinality. We discretize continuous columns into 20 quantile bins, computed on the nonzero values only, with a dedicated zero bin appended when the column has $> 5\%$ zero values (common in telemetry data where most metrics are sparse). Categorical columns are capped at 50 categories using the same top-k binning applied for the DP-VAE; values outside the top 50 are grouped into an "Other" bin. After synthesis, continuous values are reconstructed by sampling uniformly within each bin's range. This introduces quantization noise that is typically small relative to the DP noise, but can affect tail values.

We use the `MSTSynthesizer` class from the SmartNoise Synth library (v1.0.6), which wraps

the original MST implementation. We initialize with `epsilon=4.0`, `delta=1e-5`, and the default split ratio between selection and measurement budgets. The synthesizer handles the discretization, marginal selection, noise injection, PGM fitting, and sampling steps internally. For each table, we call `fit()` on the preprocessed (binned) DataFrame, then `sample(n)` to produce n synthetic rows matching the real table's row count. Synthesis for a single table takes under 10 minutes on CPU, and the full 19-table pipeline completes in approximately 90 minutes.

Like the per-table DP-SGD approach, MST synthesizes each of the 19 reporting tables independently at $\varepsilon = 4.0$ per table, $\delta = 10^{-5}$, with guid assignment proportional to each table's real join rate against the sysinfo anchor. The total privacy budget under basic composition is $19 \times 4.0 = 76.0$, identical to the per-table DP-SGD baseline. This is a conservative bound; advanced composition theorems or privacy amplification via subsampling could tighten it, but we use basic composition for simplicity and comparability.

Marginal-based methods represent the current state of the art for tabular DP synthesis (Swanberg et al. 2025) and provide the reference point against which both DP-SGD and PE should be measured.

3.4 Privacy budget summary

Table 4 compares the privacy parameters across all four synthesis methods. The wide-table DP-VAE and PE operate on a single model or mechanism, yielding a total budget equal to the per-unit budget. The per-table methods apply independent synthesis to each of the 19 reporting tables, so their total budget under basic composition is $19 \times \varepsilon = 76.0$ (we use this as a conservative bound, in the sense that it is optimizable in future work).

Table 4: Privacy parameters across synthesis methods.

Method	**Per-unit ε**	δ	**Mechanism**	**Composition**	**Total ε**
Wide-table DP-VAE	4.0	10^{-5}	DP-SGD (RDP)	single model	3.996
Per-table DP-SGD	4.0	10^{-5}	DP-SGD + Laplace	19× basic	76.0
MST	4.0	10^{-5}	Gaussian + Exp. mech.	19× basic	76.0
Private Evolution	4.0	10^{-5}	Gaussian (analytic)	T=1, single	4.0

4 Results

4.1 Evaluation metrics

We evaluate synthetic data quality by first assigning each query to one of five query categories, then routing it to the corresponding metric and pass rule. The categories are:

- Agg+Join, aggregate statistics computed after joining multiple tables on `guid` (e.g., `ChassisAgg`, `NetAsymAgg`).
- Geo/Demo, grouped summaries by geography or demographics such as country, CPU family, or generation (e.g., `XeonGeo`, `BatteryDemo`).
- Top-k, ranked outputs produced by window functions that return the highest-k items per group (e.g., `BrowserRank`, `WaitRank`).
- Pivot, wide multi-column summaries that represent joint behavior across many dimensions (e.g., `PersonaPivot`, `SleepPivot`).
- Histogram, distributional summaries over categories or bins (e.g., `BrowserHist`, `RamHist`).

For aggregate queries that produce numeric columns grouped by categorical keys, we compute the median relative error across all overlapping groups:

$$\mathrm{RE}(r,s) = |r-s|/|r|. \tag{7}$$

For queries that produce categorical distributions or histograms, we compute total variation distance between normalized real and synthetic distributions:

$$\mathrm{TV}(p,q) = \frac{1}{2}\sum_i |p_i - q_i|. \tag{8}$$

For ranked result sets, we compute Spearman's rank correlation coefficient on overlapping items. Group coverage is assessed via Jaccard similarity between real and synthetic group keys.

We route Agg+Join and Geo/Demo queries to RE, Histogram and Pivot queries to TV, and Top-k queries to Spearman ρ.

The query score is the fraction of metric columns that pass their respective threshold. A query is marked as passing when its score is at least 0.5. We report both these continuous scores (Table 15) and aggregate mean metric values (Table 12) to enable fine-grained comparison beyond binary pass/fail.

As described in Section 2.4.1, we additionally compute the *normalized error* (Equation 3), dividing each query's discrepancy by the baseline variation $|\Delta_j(D_1, D_2)|$ from a random 50/50 split of the real data (500,000 `guids` per half). Table 13 reports per-method summary statistics and Table 14 provides per-query breakdowns. We also test threshold sensitivity in Figure 8 to verify that comparative conclusions are not driven by a single cutoff choice.

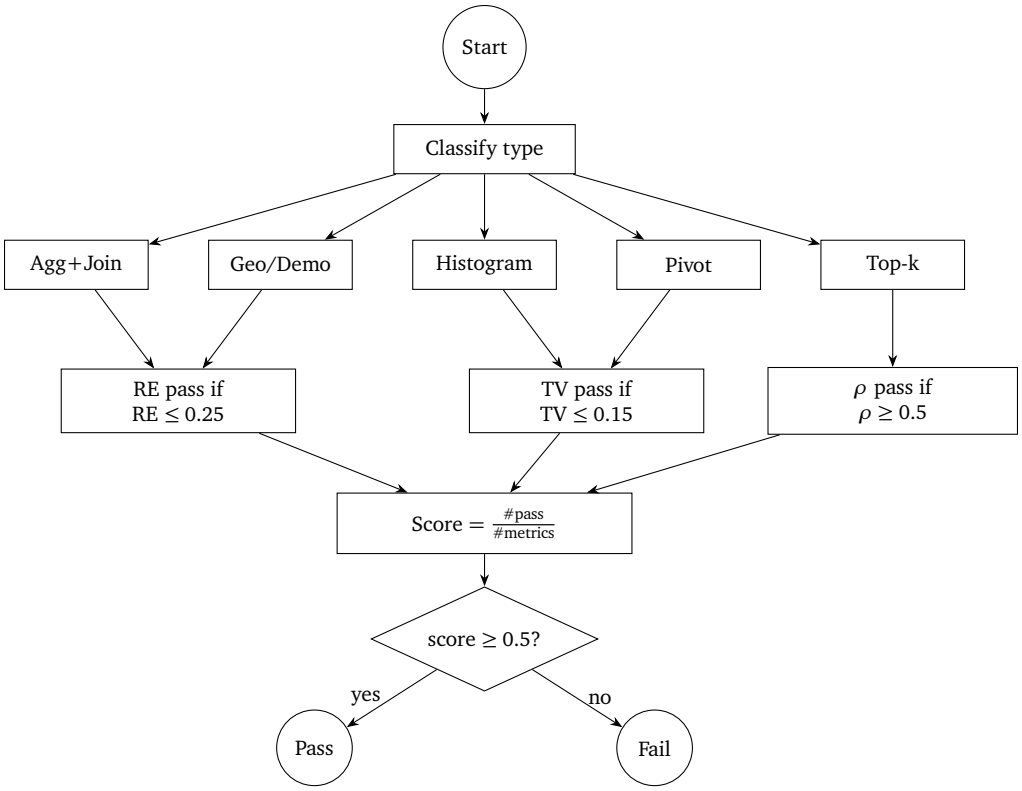

Figure 4: Metric routing and pass/fail logic used for benchmark evaluation.

4.2 DP-SGD and VAE results

We evaluate the wide-table DP-VAE synthetic data on 8 of 21 feasible benchmark queries. The remaining 13 queries require per-row reporting columns that are not represented in the wide table. The model passes 1 of 8 queries with average score 0.258 (Table 11).

To foreground per-query fidelity, Table 5 reports the 0–1 scores for the wide-table DP-VAE subset. Scores are fractions of metric-level passes. A score of 0.5 means that half of the evaluated metrics for that query satisfy threshold.

Table 5: Wide-table DP-VAE query scores on a 0–1 fidelity scale, color-coded as in Table 15: ≥ 0.5, 0.25–0.49, < 0.25.

Query codename	Type	Wide DP-VAE score
`ChassisAgg`	Agg+Join	0.167
`BatteryGeo`	Geo/Demo	0.250
`PowerGeo`	Geo/Demo	0.333
`XeonGeo`	Geo/Demo	0.250
`BrowserHist`	Histogram	0.000
`RamHist`	Histogram	0.000
`PersonaPivot`	Pivot	0.065
`BrowserRank`	Top-k	1.000

The strongest behavior is top-k categorical ranking. On `BrowserRank`, the model reaches score 1.000 and recovers 42 of 50 top-browser assignments on overlapping countries (84% categorical accuracy).

Mediocre behavior appears in several geographic queries with scores in the 0.250–0.333 range (`PowerGeo`, `XeonGeo`, and `BatteryGeo`). These are partial matches where some groups and metrics are correct, but not enough to pass.

The weakest behavior is on continuous-value workloads and join-heavy queries. In `ChassisAgg`, group overlap is moderate (Jaccard 0.571), but numeric accuracy is poor (median relative error 1161.44 for system counts, and relative error near 1.0 for the four hardware metrics). This indicates two failures at once, mismatched groups and incorrect values within overlapping groups.

Across aggregate and distribution workloads that depend on continuous magnitudes (power, temperature, frequency, network bytes, memory utilization, battery duration), relative errors commonly exceed 99%. Table 6 summarizes representative metric columns from this failure mode.

Table 6: Real vs. synthetic values for representative continuous metrics across evaluated workloads. All synthetic values are near zero.

Metric	Real	Synthetic	Rel. error
Platform power (`avg_psys_rap_watts`)	4.2914	0.0020	> 99%
Package C0 (`avg_pkg_c0`, %)	42.6308	0.0222	> 99%
Average frequency (`avg_freq_mhz`)	2,692.871	0.0070	> 99%
Temperature (`avg_temp_centigrade`)	44.7122	0.0029	> 99%
Network bytes received (`avg_bytes_received`)	7.359645×10^{16}	1.1304	> 99%
Memory used (`avg_percentage_used`, %)	42.5682	0.0000	100%
Battery duration (`avg_duration`, min)	143.9710	0.1121	> 99%

This collapse is driven by extreme zero-inflation in the wide table. Most metric columns are nonzero for only a small fraction of the 1,000,000 `guids`, because each event table covers a different subset of devices:

Table 7: Nonzero coverage per metric in the wide table. The PSYS RAP, frequency, and temperature metrics have data for fewer than 0.1% of `guids`.

Metric	**Nonzero guids**	**Sparsity**
Platform power (`avg_psys_rap_watts`)	816	99.9%
Average frequency (`avg_freq_mhz`)	613	99.9%
Temperature (`avg_temp_centigrade`)	622	99.9%
Package C0 (`avg_pkg_c0`)	8,943	99.1%
Network bytes received (`avg_bytes_received`)	37,224	96.3%
Memory used (`avg_percentage_used`)	69,552	93.0%

This failure follows from the combination of sparse data and the VAE objective. The model is trained with mean squared error on continuous columns. When 93–99.9% of values are zero, the easiest way to reduce loss is to predict values close to zero for most rows. The KL regularization term then pushes latent representations toward the prior, which makes it harder to preserve rare nonzero patterns. As a result, columns that depend on uncommon nonzero values are poorly reconstructed.

A secondary effect is inflated counts in join-heavy queries. In `ChassisAgg`, the real data returns 104 `guids` that appear in all five metric tables. The synthetic data returns more than 163,000 matches. This happens because many rows receive small positive values, so INNER JOIN conditions that should filter heavily become much less selective. This is a structural mismatch, and it reduces fidelity for multi-table aggregates.

Overall, the wide-table DP-VAE preserves one useful signal, rank ordering among frequent categorical groups. It does not preserve continuous magnitudes or strict join behavior, because rare nonzero support is not learned well (see the calibration scatter in Figure 7). This motivates the per-table strategy in the next subsection. Per-table synthesis does not preserve cross-table coupling, but it aims to improve continuous distributions.

4.3 Per-table DP-SGD results

As described in Section 3.3, this baseline synthesizes each reporting table independently at the same per-table privacy budget, using DP-VAE for anchor tables and DP histogram synthesis for the remaining tables. The goal is to improve within-table fidelity under sparse supports, at the cost of cross-table consistency.

Per-table DP-SGD passes 6 of 21 queries, with average score 0.303 and median score 0.250. Unlike wide-table DP-VAE and PE, which are evaluated on 8 queries due to wide-table reconstruction limits, per-table DP-SGD is evaluated on the full 21-query benchmark. Table 8 reports all 21 per-query scores.

Table 8: Per-table DP-SGD fidelity on all 21 benchmark queries.

Query codename	Type	Per-table score
`BlockerAgg`	Agg+Join	0.250
`BlockerDurAgg`	Agg+Join	0.000
`ChassisAgg`	Agg+Join	0.000
`DisplayAgg`	Agg+Join	0.000
`NetAsymAgg`	Agg+Join	0.143
`VendorAgg`	Agg+Join	1.000
`BatteryDemo`	Geo/Demo	0.000
`BatteryGeo`	Geo/Demo	1.000
`PowerGeo`	Geo/Demo	0.333
`XeonGeo`	Geo/Demo	0.250
`BrowserHist`	Histogram	0.333
`RamHist`	Histogram	0.500
`PersonaPivot`	Pivot	0.419
`SleepPivot`	Pivot	0.636
`BrowserRank`	Top-k	1.000
`DetectionRank`	Top-k	0.500
`FocalRank`	Top-k	0.000
`SystemRank`	Top-k	0.000
`WaitModeRank`	Top-k	0.000
`WaitRank`	Top-k	0.000
`WaitStateRank`	Top-k	0.000

This method performs best on several continuous-valued summaries. For example, in `SleepPivot`, on-time and off-time remain close to real values (RE 0.13 and 0.15). It remains weak on join-heavy queries because independently synthesized tables do not preserve cross-table correlations by `guid`.

4.4 MST results

As described in Section 3.3.1, MST is a marginal-based baseline that discretizes continuous columns, fits a private maximum-spanning-tree graphical model per table, and then samples synthetic rows from that model. Like per-table DP-SGD, it preserves table-level structure better than cross-table joins.

MST passes 6 of 21 queries, with average score 0.328 and median score 0.250. Like per-table DP-SGD, MST is evaluated on the full 21-query benchmark, while wide-table DP-VAE and PE remain limited to 8 queries. Table 9 reports all 21 per-query scores.

Table 9: MST fidelity on all 21 benchmark queries.

Query codename	Type	MST score
`BlockerAgg`	Agg+Join	0.250
`BlockerDurAgg`	Agg+Join	0.000
`ChassisAgg`	Agg+Join	0.000
`DisplayAgg`	Agg+Join	0.000
`NetAsymAgg`	Agg+Join	0.429
`VendorAgg`	Agg+Join	1.000
`BatteryDemo`	Geo/Demo	0.000
`BatteryGeo`	Geo/Demo	0.500
`PowerGeo`	Geo/Demo	0.333
`XeonGeo`	Geo/Demo	0.250
`BrowserHist`	Histogram	0.667
`RamHist`	Histogram	0.000
`PersonaPivot`	Pivot	0.194
`SleepPivot`	Pivot	0.273
`BrowserRank`	Top-k	1.000
`DetectionRank`	Top-k	1.000
`FocalRank`	Top-k	0.000
`SystemRank`	Top-k	1.000
`WaitModeRank`	Top-k	0.000
`WaitRank`	Top-k	0.000
`WaitStateRank`	Top-k	0.000

MST is strongest (albeit inconsistently) on ranking-style behavior. It attains perfect ranking agreement on several application ranking queries in the full 21-query workload. It is weaker on some continuous-value aggregates, where discretization can distort magnitudes even when category-level structure is preserved.

4.5 Private Evolution results

Table 10 reports per-query fidelity for both the vanilla and conditional PE variants on the 10 evaluable benchmark queries. The remaining 11 queries require reporting tables (display devices, foreground applications, user wait times, system moderators) that the wide-table schema does not produce.

Vanilla PE passes 1 of 10 queries with an average score of 0.137 and a median of 0.062. The sole passing query is RAM utilization (0.500), where the histogram captures the coarse distribution of memory capacity across systems. All aggregate and geographic queries score below 0.35, and several receive zero scores due to complete coverage failures on rare categorical groups.

Conditional PE (Section 3.2.2) improves to 3 of 10 passing queries with an average score of 0.306 and a median of 0.206. The three passing queries are RAM utilization (1.000), on/off/sleep summary (0.818), and Xeon network consumption (0.500). The on/off/sleep query benefits most from conditioning, rising from 0.091 to 0.818, because the per-stratum prompts embed accurate time-budget targets that the unconditioned model cannot infer from its prior alone. RAM utilization doubles from 0.500 to a perfect 1.000, and Xeon network consumption rises from 0.250 to 0.500.

Table 10: Per-query fidelity for vanilla and conditional PE on the 10 evaluable benchmark queries. Both variants operate at $\varepsilon = 4.0$, $\delta = 10^{-5}$. Conditional PE reallocates budget ($\varepsilon_{\text{agg}} = 3.5$, $\varepsilon_{\text{hist}} = 0.5$) with zCDP composition across 7 query groups.

Query	**Type**	**Vanilla PE**	**Conditional PE**	Δ
`ChassisAgg`	Agg+Join	0.167	0.000	−0.167
`BatteryDemo`	Geo/Demo	0.000	0.333	+0.333
`BatteryGeo`	Geo/Demo	0.000	0.250	+0.250
`PowerGeo`	Geo/Demo	0.000	0.000	0.000
`XeonGeo`	Geo/Demo	0.250	0.500	+0.250
`BrowserHist`	Histogram	0.333	0.000	−0.333
`RamHist`	Histogram	0.500	1.000	+0.500
`PersonaPivot`	Pivot	0.032	0.161	+0.129
`SleepPivot`	Pivot	0.091	0.818	+0.727
`BrowserRank`	Top-k	0.000	0.000	0.000
Passed (≥ 0.5)		**1/10**	**3/10**	
Average score		**0.137**	**0.306**	

The gains from conditional generation concentrate on queries whose structure aligns with the stratification. The on/off/sleep summary, stratified by CPU market code name, benefits from accurate per-stratum time-budget targets that reduce the relative error on 9 of 11 sub-metrics to below 0.19. RAM utilization improves because the conditional prompt specifies memory capacity targets that produce a tighter histogram match (TV = 0.019 vs. 0.367 for vanilla). Queries that do not align with the stratification see minimal improvement or slight regression. Browser popularity drops from 0.333 to 0.000 because the conditional variant's reduced histogram budget ($\sigma = 7.03$ vs. 1.08) weakens the selection mechanism's ability to correct the model's browser distribution bias. The chassis power query regresses from 0.167 to 0.000 because conditional PE produces zero overlapping chassis groups for the hardware power tables, a consequence of the hardware stratum being the smallest (55 records from $k = 48$ queries).

Country-stratified queries remain at zero for both variants. Even with the improved zCDP noise calibration ($\sigma \approx 24.6$ for the network group), per-country populations in the DCA corpus (typically 1 to 128 systems per country) remain below the noise floor. This represents a fundamental limitation of the DP mechanism rather than a deficiency in the generation

approach.

4.6 Cross-method comparison

Across methods, pass counts are similar for per-table DP-SGD and MST, while wide-table DP-SGD evaluates fewer queries because of schema coverage limits (Table 11). PE (conditional variant) evaluates 10 of 21 queries.

Table 11: Benchmark performance across synthesis methods. "Passed" counts queries with score ≥ 0.5.

Method	Evaluated	Passed	Pass rate	Avg score	Median score
Wide-table DP-SGD	8	1	12.5%	0.258	0.208
Per-table DP-SGD	21	6	28.6%	0.303	0.250
MST baseline	21	6	28.6%	0.328	0.250
Private Evolution (conditional)	10	3	30.0%	0.306	0.206

Table 12 reports the mean raw metric values per query type and method. For aggregate queries, the metric is median relative error (RE, lower is better); for distribution and histogram queries, total variation distance (TV, lower is better); for categorical ranking, accuracy (higher is better). These continuous values complement the binary pass rates above and reveal finer-grained differences between methods.

Table 12: Mean continuous metric values by query type. Bold indicates best (lowest error or highest accuracy) per row. Arrows indicate metric direction (↓ = lower is better, ↑ = higher is better). "—" means no queries of that type were evaluable for the method.

Query type	Metric	Wide DP-VAE	Per-table	MST	PE
Aggregate	Mean RE ↓	55.43	**0.54**	169.29	1.60
Distribution	Mean TV ↓	0.279	0.241	**0.105**	0.318
Histogram	Mean TV/RE ↓	0.744	0.444	1.000	**0.051**
Categorical ranking	Accuracy ↑	0.824	0.922	**0.941**	0.157
Numeric ranking	Spearman ρ ↑	—	0.172	**0.463**	—
Row-level	Mixed ↓	—	**1.192**	3.985	—

Per-table DP-SGD achieves the lowest aggregate RE (0.54), while MST leads on distributional fidelity (TV = 0.105) and ranking accuracy. Conditional PE achieves the lowest histogram error (0.051), driven by its near-perfect RAM utilization match, and a competitive aggregate RE (1.60) that substantially improves over the vanilla PE baseline. The high aggregate RE for MST (169.29) and wide-table (55.43) is driven by outlier queries with extreme numeric errors, suggesting these methods struggle with count preservation in joined aggregations.

4.6.1 Normalized error results

Table 13 reports the normalized error metric (Equation 3) computed by splitting the 1,000,000 anchor `guids` into two disjoint halves D_1 and D_2 (500,000 each), running all 21 benchmark queries on each half, and using $|\Delta_j(D_1, D_2)|$ as the natural-variation denominator. A normalized error of 1.0 means the synthetic data introduces exactly as much distortion as halving the real dataset; values below 1.0 indicate better-than-half-sample fidelity; values $\gg 1$ indicate synthesis error far exceeds natural variation.

Table 13: Normalized error summary across synthesis methods ($\hat{\Delta} = \Delta_j(D, \tilde{D})/|\Delta_j(D_1, D_2)|$, lower is better). Metrics with zero baseline variation are excluded (see text). Per-table DP-SGD achieves the best median normalized error (2.93), meaning its typical metric-level distortion is under 3× the natural half-sample variation. Conditional PE achieves the second-best median (3.79). Bold indicates best.

Method	Mean $\hat{\Delta}$	Median $\hat{\Delta}$	Std	Metrics
Per-table DP-SGD	14.38	**2.93**	32.78	85
PE (conditional)	14.49	3.79	30.54	60
Wide-table DP-VAE	558.67	16.42	3,252.95	51
MST baseline	1,769.84	10.76	13,503.57	85

Per-table DP-SGD achieves the best median normalized error (2.93), followed by conditional PE (3.79), MST (10.76), and wide-table DP-VAE (16.42). This ranking differs from the binary pass-rate ranking, where per-table and MST were tied at 28.6%. The normalized metric reveals that per-table DP-SGD's errors are consistently smaller relative to natural variation, and that conditional PE achieves comparable per-metric fidelity despite evaluating fewer queries. MST's high mean (1,770) is driven by extreme outliers on aggregate queries that involve multi-table joins (e.g., `SleepPivot` at 11,711× and `DisplayAgg` at 3,927×). Metrics where the baseline variation $|\Delta_j(D_1, D_2)|$ is below 10^{-8} (i.e., the two random halves produce identical values) are excluded, since they offer no natural variation to normalize against.

Table 14 provides per-query normalized errors for all four methods.

Table 14: Per-query mean normalized error ($\hat{\Delta}$, lower is better). Bold indicates best per row. "—" means the method did not produce results for that query.

Query	Type	Wide	Per-table	MST	PE
`BlockerAgg`	Agg	—	**22.7**	60.7	—
`BlockerDurAgg`	Agg	—	**2.4**	44.9	—
`ChassisAgg`	Agg	4,650.3	—	—	—
`DisplayAgg`	Agg	—	**15.7**	3,927.2	—
`NetAsymAgg`	Agg	—	**28.3**	34.2	—

Continued on next page

Query	Type	Wide	Per-table	MST	PE
VendorAgg	Agg	—	42.3	**26.1**	—
BatteryDemo	Geo	—	17.5	477.6	**10.9**
BatteryGeo	Geo	282.4	**4.1**	344.2	14.5
PowerGeo	Geo	634.1	2.8	**0.9**	3.6
XeonGeo	Geo	50.2	4.3	439.4	**3.5**
BrowserHist	Hist	115.3	91.6	**50.1**	112.8
RamHist	Hist	30.2	32.8	91.7	**1.5**
PersonaPivot	Pivot	54.9	**3.3**	32.0	9.2
SleepPivot	Pivot	—	24.3	11,710.6	**9.5**
BrowserRank	Top-k	9.0	4.0	**3.0**	43.0
DetectionRank	Top-k	—	5.4	**2.3**	—
FocalRank	Top-k	—	**1.0**	1.1	—
SystemRank	Top-k	—	43.5	**5.0**	—
WaitModeRank	Top-k	—	**0.9**	**0.9**	—
WaitRank	Top-k	—	**1.0**	**1.0**	—

The per-query breakdown reveals several patterns. First, per-table DP-SGD achieves normalized errors below 5× on 10 of 20 queries, indicating that for half of the benchmark its synthesis error is within a small multiple of natural sampling variation. Second, conditional PE achieves the lowest normalized error on 4 of its 9 evaluated queries, including RamHist (1.5×), XeonGeo (3.5×), SleepPivot (9.5×), and BatteryDemo (10.9×), confirming that conditioning on DP-noised aggregates yields strong per-metric fidelity on queries aligned with the stratification. Third, MST excels on ranking queries (WaitRank, DetectionRank) where it achieves near-unity normalized errors, confirming its strength in preserving marginal orderings. Fourth, the wide-table methods show extreme normalized errors on aggregate queries (e.g., ChassisAgg at 4,650×, PowerGeo at 634×), driven by the zero-inflation problem discussed in Section 3.1.

Figure 5 complements these tables with pass-count and average-score bars, and Figure 6 breaks average score down by query type.

Table 15: Per-query scores for all four synthesis methods, color-coded by fidelity: ≥ 0.5 (pass), 0.25–0.49, < 0.25, N/A. PE column reports the conditional variant (Section 3.2.2).

Query	Type	Wide	Per-table	MST	PE (cond.)
BlockerAgg	Agg+Join	—	0.250	0.250	—
BlockerDurAgg	Agg+Join	—	0.000	0.000	—
ChassisAgg	Agg+Join	0.167	0.000	0.000	0.000
DisplayAgg	Agg+Join	—	0.000	0.000	—
NetAsymAgg	Agg+Join	—	0.143	0.429	—

Continued on next page

Query	Type	Wide	Per-table	MST	PE (cond.)
`VendorAgg`	Agg+Join	—	1.000	1.000	—
`BatteryDemo`	Geo/Demo	—	0.000	0.000	0.333
`BatteryGeo`	Geo/Demo	0.250	1.000	0.500	0.250
`PowerGeo`	Geo/Demo	0.333	0.333	0.333	0.000
`XeonGeo`	Geo/Demo	0.250	0.250	0.250	0.500
`BrowserHist`	Histogram	0.000	0.333	0.667	0.000
`RamHist`	Histogram	0.000	0.500	0.000	1.000
`PersonaPivot`	Pivot	0.065	0.419	0.194	0.161
`SleepPivot`	Pivot	—	0.636	0.273	0.818
`BrowserRank`	Top-k	1.000	1.000	1.000	0.000
`DetectionRank`	Top-k	—	0.500	1.000	—
`FocalRank`	Top-k	—	0.000	0.000	—
`SystemRank`	Top-k	—	0.000	1.000	—
`WaitModeRank`	Top-k	—	0.000	0.000	—
`WaitRank`	Top-k	—	0.000	0.000	—
`WaitStateRank`	Top-k	—	0.000	0.000	—

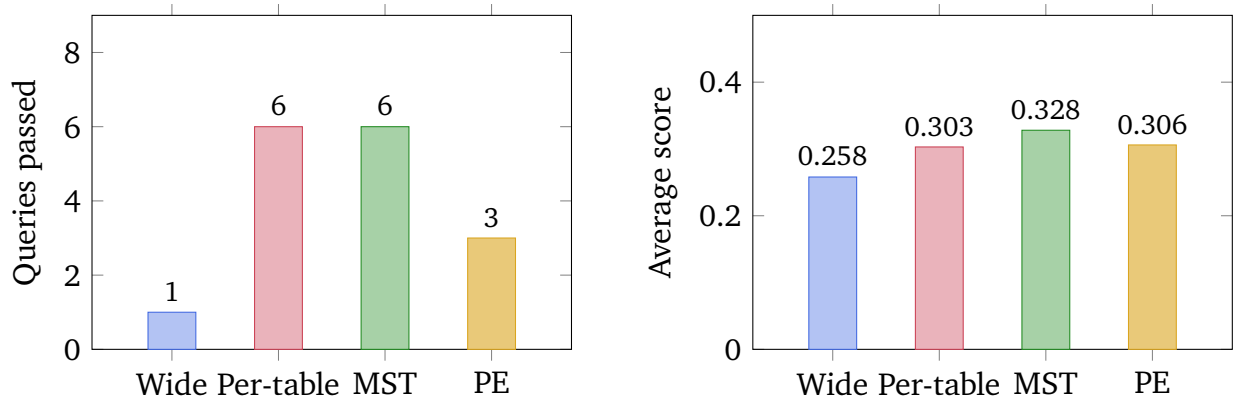

Figure 5: Left: number of queries passing (score ≥ 0.5) for each method. Right: average score across all evaluated queries. The per-table methods evaluate all 21 queries, the wide-table method evaluates 8, and PE (conditional) evaluates 10.

The per-query scores in Table 15 show that no method achieves high scores on aggregate queries involving continuous metrics, while categorical and ranking queries show more variation across methods.

Figure 6 breaks down average scores by query type. The per-table methods achieve their strongest results on pivot and histogram queries, where the independent per-table approach avoids the zero-inflation that cripples the wide-table method. Aggregate queries involving

multi-table joins remain the hardest category for all methods, with average scores below 0.3.

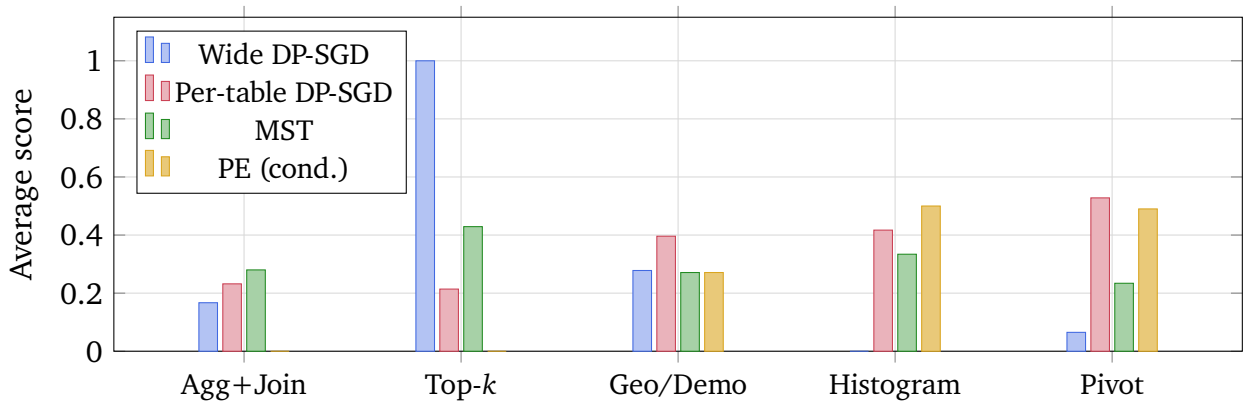

Figure 6: Average query score by query type across synthesis methods.

Figure 7 provides a cross-method calibration view. Each point is a matched group-level value from query outputs, transformed as $\log_{10}(1+x)$. Points near the diagonal indicate better calibration.

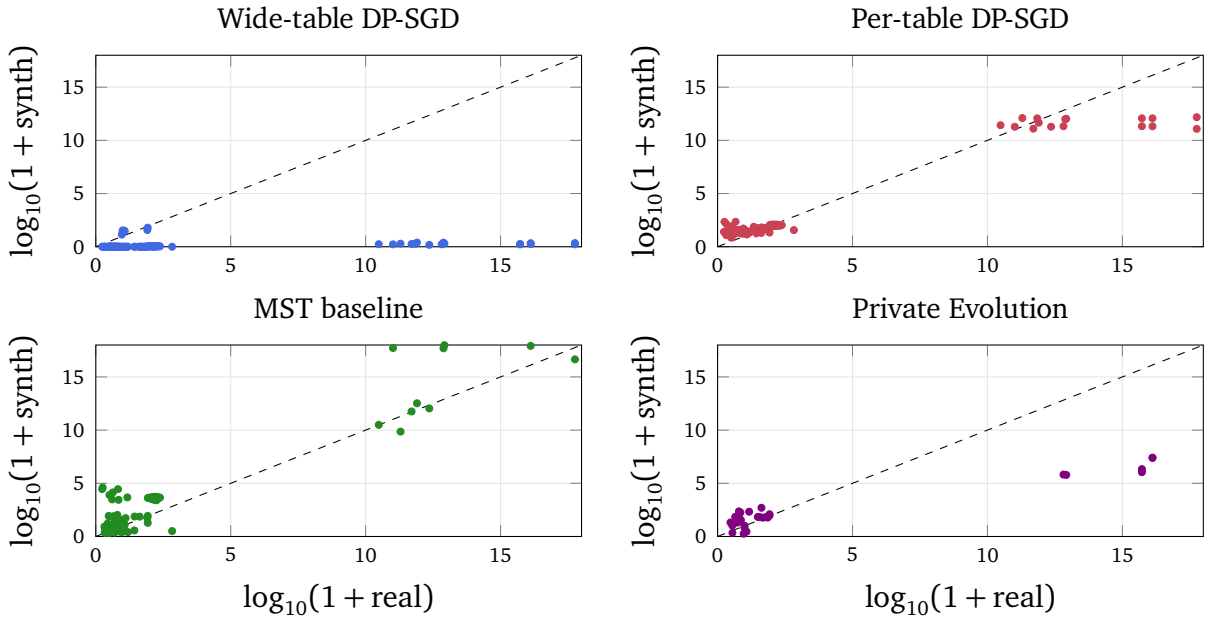

Figure 7: Calibration scatter by method using matched group-level query outputs. The dashed line is perfect calibration.

The calibration patterns differ by method. Wide-table DP-SGD shows strong collapse toward near-zero synthetic outputs, with many points concentrated at low $\log_{10}(1+\text{synth})$ even when real values are large. Per-table DP-SGD and PE both show underestimation with horizontal banding, where outputs plateau at a few repeated synthetic levels rather than

tracking continuous increases in the real values. MST has wider spread and several outliers, including high-value deviations, but its points follow the diagonal more closely across a broader range. This is consistent with MST being noisier pointwise while remaining relatively calibrated overall.

4.7 Additive experiments

We ran a second round of additive experiments to test whether lightweight interventions can improve utility without changing any of the original methods.

The first intervention was post-processing on top of existing synthetic reporting tables. We enforced numeric constraints and canonicalized selected categorical values to real-data vocabularies. This intervention produced no measurable gain. Per-table DP-SGD, MST, and PE all had unchanged pass counts and unchanged average scores.

The second intervention was workload routing. This intervention does not generate new synthetic data. It selects query outputs from the method that performs better for that query type. We routed aggregate and histogram queries to per-table DP-SGD, then routed distribution and ranking queries to MST. This lifted the benchmark from 6 of 21 passed queries to 8 of 21 with an average score of 0.404.

Table 16: Additive follow-up experiment results. The routing policy combines existing query outputs by query type and improves overall benchmark utility.

Run	Evaluated	Passed	Pass rate	Avg score
Per-table DP-SGD baseline	21	6	28.6%	0.303
MST baseline	21	6	28.6%	0.328
Postprocessed per-table DP-SGD	21	6	28.6%	0.303
Postprocessed MST	21	6	28.6%	0.328
Hybrid routed outputs	21	8	38.1%	0.404

The conditional PE variant described in Section 3.2.2 subsumes the stratified generation experiments originally planned as additive interventions. Its methodology (zCDP composition across 7 query groups, budget rebalancing with $\varepsilon_{\text{agg}} = 3.5$ and $\varepsilon_{\text{hist}} = 0.5$, and per-stratum prompting from DP-noised aggregates) is reported in full in the PE results section above.

5 Discussion

We frame our discussion around the five research questions we posed in Section 2.5.

> *(1) Under matched (ε, δ), which method achieves higher benchmark scores, and which query types show the largest gap?*

At the benchmark level, no single method dominates all query families. The per-table methods produce the strongest overall utility and pass a larger share of the workload than the wide-table and training-free runs in this study, at the sacrifice of a substantially weaker privacy budget. Between those two per-table methods, MST has a small edge in average score, while per-table DP-SGD remains competitive and stronger on selected continuous summaries. The broad pattern is that ranking and distribution-oriented queries are more recoverable than join-intensive queries post-synthesis. This stipulates that method choice should be treated as workload-dependent rather than universal.

> *(2) Does error compound across multi-table joins, or is joint structure preserved adequately?*

Error does compound in multi-table queries, and this is the clearest structural result in the project. Independent per-table synthesis improves realism at the local level, but weakens coherence between tables at join time. The wide-table approach preserves a shared client-level representation and therefore retains some relational structure, yet it introduces sparsity that degrades many continuous outputs. In practical terms, the study shows a tradeoff between join consistency and sparse-feature fidelity. The current pipelines do not remove that tradeoff.

> *(3) Which method better preserves minority class frequencies (prevalence $<$ 5%)?*

`BrowserRank` demonstrates that all methods preserve dominant categorical associations. Per-table DP-SGD achieves 47 of 50 correct top browsers per country, and the wide-table DP-VAE achieves 42 of 50. For rare categories, per-table DP-SGD inflates minority chassis types due to the top-50 binning and uniform noise in DP-SGD, while MST underrepresents them (generating too few entries for rare combinations). Both methods underperform on queries involving rare process names or application-specific rankings. The implication is that analysis of minority groups should include targeted diagnostics beyond aggregate pass and fail rates.

> *(4) Do classifiers trained on synthetic data match performance of classifiers trained on real data?*

This question remains open in the current report. The benchmark queries are analytical (aggregate statistics, rankings, distributions), not predictive, so training downstream classifiers on the synthetic data was not a natural fit for this evaluation framework. A direct

comparison would require defining a classification task on the DCA data (e.g., predicting chassis type from usage patterns), training models on real versus synthetic data, and comparing test accuracy. We defer this to future work.

(5) What are the wall-clock and resource tradeoffs for each method?

There are clear differences in resource costs between methods. The wide-table DP-VAE has no external API bill, but it is the most compute intensive option and requires 360 minutes on CPU (20 epochs, 1M rows, 307 features). Per-table synthesis also avoids APIs and completes in approximately 90 minutes total (two VAEs at ∼30 minutes each, plus histogram synthesis for 17 smaller tables). MST is the lightest local compute path in this study and runs in under 10 minutes per table. PE is the only method with explicit out-of-pocket cost because it depends on a paid foundation-model API. Vanilla PE completes in approximately 52 minutes (2,000 async structured-output calls to gpt-5-mini generating 10,000 candidates, followed by DP histogram selection). Conditional PE completes in approximately 37 minutes with fewer total API calls because per-stratum generation produces a tighter candidate pool. Both PE variants cost under $5 USD per run at current API pricing.

5.0.1 Overall interpretation

The PE results confirm that foundation models import their own distributional priors rather than learning the target distribution. Vanilla PE passes only 1 of 10 evaluable queries with an average score of 0.137. Conditional PE, which guides generation using DP-noised aggregate statistics (Section 3.2.2), improves to 3 of 10 passed queries with an average score of 0.306. The gains concentrate on queries whose structure aligns with the stratification, particularly RAM utilization (1.000) and the on/off/sleep summary (0.818), while country-level queries remain at zero because per-country populations fall below the DP noise floor. Although structured output constraints eliminate out-of-vocabulary categorical values, the distributional mismatch between the foundation model's prior and the real data still reduces the effective candidate pool for the nearest-neighbor histogram. This aligns with Swanberg et al.'s finding that LLMs capture 1-way marginals reasonably but are inaccurate on k-way marginals.

Several mitigation strategies warrant investigation.

- A two-stage model in which a Bernoulli model predicts zero vs. nonzero per column, followed by a conditional model for nonzero values only.
- Relational DP synthesis methods that handle multi-table structure directly, eliminating the zero-inflation problem while preserving cross-table correlations.
- Method ensembling, using per-table DP-SGD for queries requiring continuous accuracy and MST for queries requiring ranking preservation, selecting per query type.
- Post-processing the synthetic data with mode patching and constraint enforcement, as proposed by recent work on model-agnostic DP post-processing.

5.1 Sensitivity to evaluation thresholds

Figure 8 reports how pass rates change as the relative error, total variation, and Spearman ρ thresholds vary. The analysis confirms that our findings are not artifacts of a particular threshold choice.

The relative error threshold has the largest effect. Per-table DP-SGD's pass rate increases from 14.3% at RE $\leq$ 0.05 to 57.1% at RE $\leq$ 1.0, indicating many queries produce synthetic values in the right order of magnitude but outside the strict 25% tolerance. MST is more robust to strict thresholds. It starts at 23.8% at RE $\leq$ 0.05 (compared to 14.3% for per-table), reflecting its strength on categorical distributions where exact values are less important than distributional shape. The wide-table method is flat at 12.5% until RE $\leq$ 1.0, at which point it jumps to 75%, confirming that the continuous metric failure is total (relative errors exceed 99%) rather than marginal.

Total variation and Spearman ρ thresholds have minimal effect on pass rates across all methods. TV sensitivity is limited because the distribution queries either match closely (TV < 0.10) or fail badly (TV > 0.5), with few queries in the boundary region. Rank correlation sensitivity is limited because MST achieves perfect $\rho = 1.0$ on its passing ranking queries while both methods score near zero on failing ones, leaving no queries sensitive to the threshold.

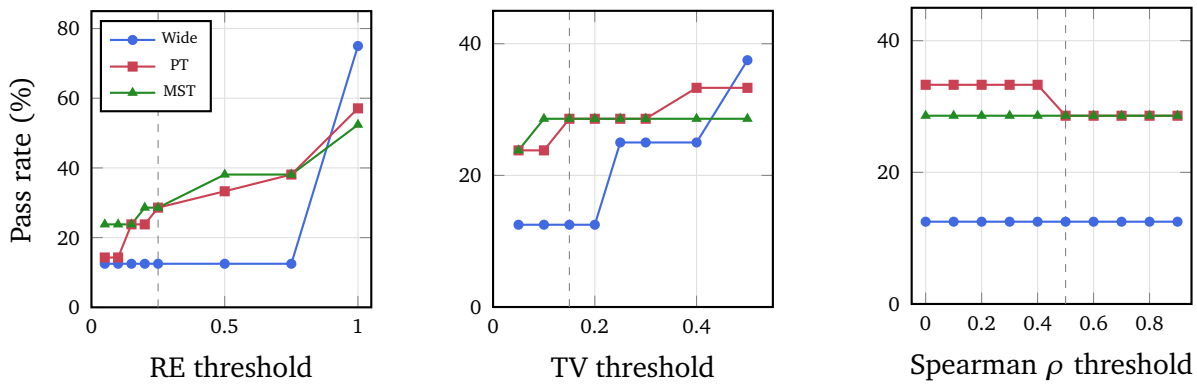

Figure 8: Pass rate sensitivity to evaluation thresholds for the three training-based methods. Left: relative error threshold (TV = 0.15, ρ = 0.5 fixed). Center: total variation threshold (RE = 0.25, ρ = 0.5 fixed). Right: Spearman ρ threshold (RE = 0.25, TV = 0.15 fixed). Dashed gray lines indicate the default thresholds.

6 Conclusion

We presented an evaluation framework for differentially private synthetic data generation on Intel's DCA telemetry corpus, implementing four synthesis approaches, namely a wide-table DP-VAE trained with DP-SGD, per-table DP histogram synthesis, MST marginal-based synthesis, and Private Evolution via foundation model APIs.

All four methods have completed evaluation against the 21-query benchmark. Under binary pass-rate scoring, per-table DP-SGD and MST each pass 6 of 21 queries (28.6%), the wide-table DP-VAE passes 1 of 8 (12.5%), and conditional PE passes 3 of 10 (30.0%). The normalized error metric (Section 2.4.1), which calibrates each query's synthesis error against the natural variation from a 50/50 data split, provides further differentiation across all four methods: per-table DP-SGD achieves a median normalized error of 2.93×, meaning its typical distortion is under 3 times the variation expected from halving the real dataset, followed by conditional PE at 3.79×, MST at 10.76×, and wide-table DP-VAE at 16.42×. Per-table DP-SGD achieves normalized errors below 5× on 10 of 20 queries, while conditional PE achieves the lowest normalized error on 4 of its 9 evaluated queries, including RAM utilization (1.5×), Xeon network consumption (3.5×), sleep pivot (9.5×), and battery demographics (10.9×). MST excels specifically on ranking queries (achieving near-unity normalized errors on `WaitRank` and `WaitModeRank`) but suffers extreme errors on multi-table aggregate joins.

Two structural findings emerge. First, wide-table sparsity, not the choice of synthesis algorithm, is the dominant failure mode. The wide table has 93–99.9% zero values in metric columns, and the VAE collapses these to their mode. Second, per-table independence destroys cross-table correlations. The 5-way chassis join produces empty or random results under both per-table methods, while the wide-table approaches (DP-VAE and PE) at least preserve some cross-table structure. Conditional PE partially addresses the sparsity problem through per-stratum prompting, achieving the lowest histogram error (0.051) and a competitive aggregate relative error (1.60) among all methods.

Our PE results are consistent with Swanberg et al. (2025). Vanilla PE underperforms the training-based methods, but conditional PE demonstrates that guiding generation with DP-noised aggregates can close much of the gap, particularly on queries whose structure aligns with the stratification. The broader lesson is that structure in the data representation and workload alignment matter more than model class alone.

The most promising next step is relational DP synthesis that models linked tables directly, so sparsity and cross-table dependence are handled in the same mechanism. In parallel, a practical benchmark strategy is to combine methods by query type, using each method where it is most reliable while preserving a single, auditable privacy accounting framework.

References

Abadi, Martin, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. "Deep learning with differential privacy." In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. [Link]

Balle, Borja, and Yu-Xiang Wang. 2018. "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising." In *Proceedings of the 35th International Conference on Machine Learning*. PMLR. [Link]

Bun, Mark, Cynthia Dwork, Guy N. Rothblum, and Salil Vadhan. 2016. "Concentrated Differential Privacy." [Link]

Dwork, Cynthia, and Aaron Roth. 2014. "The Algorithmic Foundations of Differential Privacy." *Foundations and Trends® in Theoretical Computer Science* 9 (3–4): 211–407. [Link]

Ghalebikesabi, Sahra, Leonard Berrada, Sven Gowal, Ira Ktena, Robert Stanforth, Jamie Hayes, Soham De, Samuel L Smith, Olivia Wiles, and Borja Balle. 2023. "Differentially private fusion models generate useful synthetic images." *arXiv preprint arXiv:2302.13861*

Lin, Zinan, Sivakanth Gopi, Janardhan Kulkarni, Harsha Nori, and Sergey Yekhanin. 2025. "Differentially Private Synthetic Data via Foundation Model APIs 1: Images." [Link]

McKenna, Ryan, Gerome Miklau, and Daniel Sheldon. 2021. "Winning the NIST Contest: A Scalable and General Approach to Differentially Private Synthetic Data." *Journal of Privacy and Confidentiality* 11 (3). [Link]

McKenna, Ryan, Daniel Sheldon, and Gerome Miklau. 2019. "Graphical-model based estimation and inference for differential privacy." In *Proceedings of the 36th International Conference on Machine Learning*. PMLR. [Link]

McSherry, Frank, and Kunal Talwar. 2007. "Mechanism design via differential privacy." In *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*. IEEE. [Link]

OpenDP Contributors. 2024. "SmartNoise SDK." [Link]

Swanberg, Marika, Ryan McKenna, Edo Roth, Albert Cheu, and Peter Kairouz. 2025. "Is API Access to LLMs Useful for Generating Private Synthetic Tabular Data?." [Link]

Xie, Chulin, Zinan Lin, Arturs Backurs, Sivakanth Gopi, Da Yu, Huseyin A Inan, Harsha Nori, Haotian Jiang, Huishuai Zhang, Yin Tat Lee, Bo Li, and Sergey Yekhanin. 2024. "Differentially Private Synthetic Data via Foundation Model APIs 2: Text." [Link]

Zhang, Aston, Zachary C. Lipton, Mu Li, and Alexander J. Smola. 2023. *Dive into Deep Learning*. Cambridge University Press. [Link]

A Project proposal

A.1 Abstract

Differentially private synthetic data generation enables the release of realistic datasets without exposing individual records. Two paradigms dominate current research: *training-based* methods such as differentially private stochastic gradient descent (DP-SGD), which inject noise during model optimization, and *training-free* methods such as Private Evolution (PE), which achieve privacy guarantees through inference-only access to pre-trained foundation models.

We propose to implement and rigorously compare both approaches on Intel's Driver and Client Applications (DCA) telemetry corpus, a multi-table dataset comprising system metadata, power consumption, application usage, and browsing behavior across thousands of Windows clients. Our evaluation centers on a benchmark of 22 analytical SQL queries representative of real business intelligence workloads, measuring whether synthetic data can substitute for real data in production analytics pipelines. Under matched privacy budgets, we assess query fidelity, statistical preservation, downstream utility, and computational cost to determine whether training-free methods can match training-based approaches while offering reduced implementation complexity.

A.2 Motivation

Organizations routinely collect detailed telemetry from their products that drive critical business decisions. Engineers use it to diagnose failures, product teams analyze usage trends, and analysts extract insights that shape product roadmaps. However, the same granularity that makes telemetry valuable also makes it sensitive. Browsing patterns, work schedules, and device fingerprints can re-identify specific individuals even after conventional anonymization.

Privacy regulations such as GDPR and CCPA, along with institutional policies, increasingly restrict how such data can be stored, shared, and analyzed. The resulting tension between data utility and privacy protection motivates the development of *synthetic data generation*, which produces artificial datasets that preserve the statistical properties necessary for analysis while providing formal guarantees that no individual's information can be recovered.

Two paradigms have emerged for generating synthetic data with rigorous privacy guarantees. *Training-based methods* optimize generative models under constraints that guarantee privacy (by adding noise during the training process to effectively "anonymize"). *Training-free methods* leverage pre-trained foundation models (e.g., ChatGPT, Claude, Gemini), achieving privacy through carefully designed queries rather than private optimization. Both approaches have demonstrated success in isolation, yet no comprehensive comparison

exists under controlled experimental conditions with realistic analytical workloads.

This project provides that comparison. We implement both paradigms and evaluate them on a real telemetry corpus using a benchmark of 22 SQL queries representative of production analytics, measuring which approach better preserves query fidelity under equivalent privacy budgets.

A.2.1 Technical context

Differential privacy (DP) provides the mathematical foundation for our privacy guarantees. Informally, a mechanism is differentially private if its output looks nearly the same whether or not any single individual's data is included. Formally, a mechanism M satisfies (ε, δ)-DP if for neighboring datasets $D \sim D'$ differing by one record:

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{9}$$

The parameter ε controls the privacy-utility tradeoff, as smaller values mean stronger privacy but noisier outputs.

Two methodologies dominate differentially private synthetic data generation:

1. *Training-based (DP-SGD):* Train a generative model while adding noise to gradients at each step (Abadi et al. 2016). This is effective, but is computationally expensive and requires careful hyperparameter tuning.
2. *Training-free (Private Evolution):* Use API access to pre-trained foundation models and treat them as black boxes, iteratively selecting and mutating candidates via differentially private histograms (Lin et al. 2025). This is much easier to deploy, but is theoretically more abstract.

Our central research question: *can training-free methods match the statistical fidelity of training-based approaches on real analytical workloads?*

A.3 Data and problem statement

A.3.1 Intel DCA telemetry data

Our investigation involves the Intel Driver and Client Applications (DCA) telemetry dataset, a large-scale collection of system-level signals from Windows client machines. Telemetry data is invaluable for product monitoring, performance optimization, and failure detection, yet contains sensitive behavioral patterns that could re-identify individual users. We describe the dataset structure and the analytical workload that synthetic data must preserve.

The DCA telemetry corpus comprises ≈ 30 interrelated tables, organized around a globally unique identifier (`guid`) assigned to each client system. Tables are indexed by `guid` and, where applicable, a date column (`dt`), The schema spans several categories:

- *Client metadata:* The `system_sysinfo_unique_normalized` table provides static attributes, like chassis type (notebook, desktop, 2-in-1, tablet), country, OEM, RAM capacity, processor family/generation, graphics card, screen size, and a derived *persona* classification (Gamer, Office/Productivity, Content Creator, etc.).
- *Power and thermal:* Multiple tables capture physical instrumentation:
  - `system_hw_pkg_power`: Processor power draw (mean, max) in Watts
  - `system_psys_rap_watts`: Total system power across AC/DC states
  - `system_pkg_temp_centigrade`: Processor temp. distributions by power state
  - `system_pkg_c0`: C0 state residency (percentage of time fully active)
  - `system_pkg_avg_freq_mhz`: Clock frequency statistics
- *Battery and mobility:* The `system_batt_dc_events` table summarizes battery utilization (duration on DC, battery percentage, power cycle frequency). Daily durations for the on/off/sleep states are also recorded.
- *Application behavior:* Foreground usage in `system_frgnd_apps_types` and a daily summary (process names, categories, focal time). The `system_userwait` table measures wait times for application starts.
- *Web browsing:* The `system_web_cat_pivot_*` family decomposes browsing across 28 categories (social, productivity, entertainment, gaming, etc.) by duration, page loads, and domain counts.
- *Network and memory:* The `system_network_consumption` table summarizes bytes sent/received; `system_memory_utilization` provides RAM usage statistics.
- *Display:* The `system_display_devices` table records connected displays (connection type, resolution, refresh rate, duration by AC/DC state).

The practical utility of synthetic telemetry data is determined by its ability to support *real analytical workloads*. We operationalize this through a benchmark suite of 22 SQL queries representative of business intelligence tasks performed on the DCA corpus. These queries, developed by Intel analysts, span several classes, where the following are examples of such:

1. *Aggregate statistics with joins* involve weighted statistics across multiple tables joined on `guid`:

```sql
-- Average platform power, C0 residency, frequency, and
 ↪  temperature by chassis type
SELECT chassistype, COUNT(DISTINCT guid) AS n_systems,
       SUM(nrs * avg_psys_rap_watts) / SUM(nrs) AS avg_power,
       SUM(nrs * avg_pkg_c0) / SUM(nrs) AS avg_c0, ...
FROM system_sysinfo_unique_normalized a
INNER JOIN system_psys_rap_watts b ON a.guid = b.guid
INNER JOIN system_pkg_c0 c ON a.guid = c.guid ...
GROUP BY chassistype;
```

2. *Ranked top-k queries* involve window functions for ranked lists:

```sql
-- Top 10 applications per category by focal screen time
SELECT app_type, exe_name, avg_focal_sec_per_day,
       RANK() OVER (PARTITION BY app_type
                    ORDER BY avg_focal_sec_per_day DESC) AS rank
FROM ( ... ) WHERE rank ≤ 10;
```

3. *Geographic/demographic breakdowns* involve segmentation by country, processor generation, OEM, or persona:

```sql
-- Battery usage by country
SELECT countryname_normalized, COUNT(DISTINCT guid),
       AVG(num_power_ons), AVG(duration_mins)
FROM system_batt_dc_events a
INNER JOIN system_sysinfo_unique_normalized b ON a.guid = b.guid
GROUP BY countryname_normalized HAVING COUNT(DISTINCT guid) > 100;
```

4. *Histograms and distributions* involve binned aggregations testing distributional shape preservation:

```sql
-- RAM utilization by memory capacity
SELECT sysinfo_ram / 1024 AS ram_gb, COUNT(DISTINCT guid),
       ROUND(SUM(nrs * avg_percentage_used) / SUM(nrs)) AS avg_pct
FROM system_memory_utilization
WHERE avg_percentage_used > 0
GROUP BY sysinfo_ram ORDER BY sysinfo_ram;
```

5. *Complex multi-way pivots* involve high-dimensional joint distributions, e.g., web category usage percentages by persona across 28 browsing categories.

Let $\mathcal{Q} = \{q_1, \ldots, q_{22}\}$ denote our SQL query benchmark. Each query q_j maps a database instance to a result set $q_j(D) \in \mathcal{R}_j$, where $\mathcal{R}_j$ may be a scalar, vector, or table. The *query discrepancy* for synthetic data $\tilde{D}$ is:

$$\Delta_j(D, \tilde{D}) = d_j(q_j(D), q_j(\tilde{D})), \tag{10}$$

where d_j is a distance metric appropriate to the result type (relative error for scalars, Spearman's ρ for rankings, total variation for histograms). The aggregate benchmark score is:

$$\text{Score}(\tilde{D}) = \frac{1}{22} \sum_{j=1}^{22} \mathbf{1}[\Delta_j(D, \tilde{D}) \leq \tau_j], \tag{11}$$

where τ_j is a query-specific tolerance. A synthetic dataset "passes" the benchmark if it achieves high scores, indicating analysts could substitute $\tilde{D}$ for D without materially affecting conclusions.

A.3.2 Problem statement

Let $D = \{x_1, \ldots, x_n\}$ be a private dataset. Two datasets D, D' are *neighbors* if they differ by one record. A mechanism M satisfies (ε, δ)-differential privacy if for all neighbors $D \sim D'$ and all outputs S:

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{12}$$

Our objective is to construct a synthetic data mechanism M producing $\tilde{D} = M(D)$ that satisfies (ε, δ)-DP while preserving utility (i.e., the synthetic distribution should approximate the real distribution sufficiently for downstream analytics).

Both paradigms exhibit known limitations:

- *DP-SGD limitations:*
  - Gradient clipping bias disproportionately affects rare classes
  - Noise accumulation degrades convergence over many iterations
  - Complex hyperparameter tuning (batch size, learning rate, clip norm, noise multiplier)
  - High computational cost (see Ghalebikesabi et al. (2023))
- *Private Evolution limitations:*
  - Utility depends on foundation model quality, and some may underperform on specialized tabular data
  - Nearest-neighbor histograms assume embedding similarity correlates with distributional similarity
  - Limited theoretical understanding compared to optimization-based methods

Given these limitations and our 22-query SQL benchmark, we investigate:

1. Under matched (ε, δ), which method achieves higher scores on the SQL query benchmark? Which query types exhibit the largest discrepancy?
2. Does error compound across multi-table joins, or do synthetic data preserve joint distributions adequately?
3. Which method better preserves minority class frequencies ($< 5\%$ prevalence)?
4. Do classifiers trained on synthetic data achieve comparable accuracy to those trained on real data?
5. What are the wall-clock time and resource requirements for each method?

These questions remain open because prior work evaluates methods in isolation, on different datasets, under incomparable conditions. Our contribution is a controlled comparison on a concrete analytical workload.

A.3.3 Approach

For training-based synthesis, we will implement a differentially private generative model using PyTorch and the Opacus library, with privacy guarantees via DP-SGD (per-sample gradient clipping and calibrated noise injection). For training-free synthesis, we will implement Private Evolution using black-box API access to foundation models, achieving privacy through differentially private nearest-neighbor histograms. Both methods will be evaluated under matched privacy budgets on our 22-query SQL benchmark, comparing query fidelity, statistical preservation, downstream utility, and computational cost.

A.3.4 Expected outputs

1. *Technical report:* Implementation details, privacy analysis, query-by-query benchmark results, statistical fidelity analysis, and practical recommendations.
2. *SQL benchmark suite:* The 22-query benchmark with natural language specifications, SQL code, tolerance thresholds, evaluation scripts, and baseline results.
3. *Differentially private synthetic datasets:* Two synthetic versions of the DCA corpus under matched (ε, δ) (one from DP-SGD, one from PE) with documented privacy guarantees.
4. *Project website:* Public-facing summary with visualizations, query-level comparisons, and deployment guidelines.
5. *Open-source implementations:* Reproducible code for preprocessing, training, generation, and evaluation.

B Contributions

Mehak Kapur ran and validated DuckDB queries to ingest the Parquet datasets. They helped construct the unified wide training table. They implemented and executed the DP-SGD VAE pipeline to generate synthetic data. They produced evaluation outputs and visualizations comparing real versus synthetic distributions. They contributed to report writing.

Hana Tjendrawasi ran and validated DuckDB queries to ingest the Parquet datasets. They helped construct the unified wide training table by aligning schemas across sources. They led project logistics. They supported the DP-SGD workflow through preprocessing and experiment setup. They assisted with reviewing evaluation metrics and results. They contributed to report writing.

Jason Tran built the 19 reporting tables from raw telemetry data via DuckDB aggregation scripts and parsed the 24 queries. They designed and implemented the wide-table vs. per-table synthesis strategy, the PE pipeline, and the MST baseline workflow for tabular comparison. They supported the DP-SGD workflow through implementation. They implemented benchmark execution, decomposition, and evaluation across real and synthetic outputs, integrated methods and findings into the report. They contributed to report writing.

Phuc Tran performed exploratory data analysis on raw telemetry tables. They conducted schema verification and column mapping between raw and reporting tables.